

## Highlights

### **FlashSpread: IO-Aware GPU Simulation of Non-Markovian Epidemic Dynamics via Kernel Fusion**

Heman Shakeri, Behnaz Moradi-Jamei, Aram Vajdi, Ehsan Ardjmand

- First GPU-accelerated non-Markovian epidemic simulator; scales to  $N=10^8$ .
- Fused Triton kernel achieves 8.09 Giga-NUPS on A100,  $217\times$  over CPU tau-leaping.
- Degree-aware CSR auto-dispatch delivers  $4.5\times$  on scale-free graphs.
- Source-node  $O(N)$  approximation: age-dependent shedding at negligible cost.
- $\sim 6\%$  structural-bias floor on peak infection ( $\sim 7\%$  on final attack rate) vs. exact Gillespie, robust across tested step sizes.

# FlashSpread: IO-Aware GPU Simulation of Non-Markovian Epidemic Dynamics via Kernel Fusion

Heman Shakeri<sup>a,\*</sup>, Behnaz Moradi-Jamei<sup>b</sup>, Aram Vajdi<sup>a</sup> and Ehsan Ardjmand<sup>c</sup>

<sup>a</sup>School of Data Science, University of Virginia, Charlottesville, 22904, VA, USA

<sup>b</sup>Department of Mathematics & Statistics, James Madison University, Harrisonburg, 22807, VA, USA

<sup>c</sup>Ohio University, Athens, 45701, OH, USA

## ARTICLE INFO

### Keywords:

Stochastic simulation  
Non-Markovian renewal processes  
GPU computing  
Epidemic modeling  
Tau-leaping  
Complex networks  
Kernel fusion  
CUDA Graphs

## ABSTRACT

Non-Markovian (renewal) epidemic simulation on multi-million-node contact networks is essential for realistic forecasting under general age-dependent holding-time distributions (log-normal, Weibull, Erlang, and similar), but the age-dependent hazard forces dense per-step updates that render the sparse event-queue strategies of standard CPU methods ineffective. We present FLASHSPREAD, a GPU framework that consolidates the per-step renewal pipeline (CSR traversal, numerically stable erfcx-based hazard evaluation, Bernoulli tau-leaping, state transition, and next-step infectivity write-back) into a single fused Triton kernel whose intermediates never leave streaming-multiprocessor registers, with block-scalar skips that preserve CUDA Graph capture and a degree-aware CSR dispatch (thread / warp / edge-merge) that keeps the peak throughput on scale-free graphs. On an NVIDIA A100 the fused CUDA-Graph engine reaches 8.09 Giga-NUPS at  $N = 10^6$  on a uniform-degree graph, a 217 $\times$  strict hardware speedup over optimised CPU tau-leaping at the same  $N$ ; on a Barabási-Albert graph of the same size the merge-based dispatch recovers 4.5 $\times$  (0.45  $\rightarrow$  2.0 Giga-NUPS) over the default kernel, and the framework scales to  $N = 10^8$  on a single A100 (40 GB), with a mixed-precision storage path that extends the L2-reachable scale by roughly 3 $\times$  and delivers a 1.15 $\times$  throughput lift at the far bandwidth-bound end. Validation against an exact non-Markovian Gillespie reference shows a structural-bias floor of  $\sim 6\%$  on peak infection and  $\sim 7\%$  on final attack rate that does not detectably decrease as  $\epsilon \rightarrow 0$  across two decades of tolerance, comfortably within typical epidemiological parameter uncertainty. Code: <https://github.com/Shakeri-Lab/FlashSpread>.

## 1. Introduction

Stochastic spreading processes on networks (epidemics, information cascades, product adoption, malware propagation) pose fundamental modeling and control challenges (Pastor-Satorras, Castellano, Van Mieghem and Vespignani, 2015; Kiss, Miller and Simon, 2017; Barrat, Barthélemy and Vespignani, 2008). The underlying system is a high-dimensional continuous-time Markov chain whose state space grows exponentially with network size, rendering exact analytical solutions intractable for realistic systems (Anderson and May, 1991; Keeling and Rohani, 2008).

Two distinct computational regimes arise depending on the temporal physics. In *Markovian processes* (e.g., traditional SIS, SIR), transition rates depend only on current state and external control inputs; rates remain constant between events, enabling sparse incremental updates. In *stochastic renewal processes* (e.g., SEIR with age-dependent incubation), transition rates depend on continuous holding times (ages) and change continuously even in the absence of events, requiring dense synchronous updates.

Markovian dynamics are the special case of renewal dynamics in which every holding-time distribution is exponential; throughout this paper we use *Markovian* for that special case and *non-Markovian (renewal)* for the general age-dependent case that motivates the framework. Computationally the two regimes require *fundamentally opposite optimisation strategies* — sparse event-driven updates for memoryless dynamics versus dense time-stepping for age-dependent hazards — which motivates our dual-engine architecture.

The reliance on Markovian processes in epidemiological modeling, while mathematically convenient, imposes a “memoryless” assumption that contradicts the biological reality of pathogen transmission (Vajdi, Cohnstaedt, Noronha, Mitzel, Wilson and Scoglio, 2023). Standard Markovian models assume dwell times follow exponential distributions,

\*Corresponding author

 [hs9hd@virginia.edu](mailto:hs9hd@virginia.edu) (H. Shakeri)

ORCID(s): 0000-0002-9891-5748 (H. Shakeri)

whose monotonically decreasing density implies a constant hazard rate: an infected individual is equally likely to recover at any moment, regardless of how long they have been infected. In contrast, empirical data from outbreaks such as COVID-19 demonstrate that critical epidemiological parameters (incubation period, serial interval, infectious duration) follow peaked, unimodal distributions (e.g., Weibull, log-normal, Erlang) (Sanche, Lin, Xu, Romero-Severson, Hengartner and Ke, 2020; Backer, Klinkenberg and Wallinga, 2020; Lauer, Grantz, Bi, Jones, Zheng, Meredith, Azman, Reich and Lessler, 2020), for which transition probability is negligible immediately after state entry and peaks after a characteristic delay. Neglecting this temporal structure leads to significant biases in predicting epidemic peaks and basic reproduction numbers, since distributions with identical means but differing variances yield substantially different trajectories (Vajdi et al., 2023; Sherborne, Miller, Blyuss and Kiss, 2018; Feng, Cai, Tang and Lai, 2019). Van Mieghem and van de Bovenkamp (2013) showed that non-Markovian infection dynamics can dramatically alter epidemic thresholds compared to Markovian approximations, and for practical decision-making regarding contact tracing and quarantine the precise timing of infectivity relative to symptom onset is paramount (Vajdi et al., 2023; He, Lau, Wu, Deng, Wang, Hao, Lau, Wong, Guan, Tan et al., 2020).

Despite these limitations in biological realism, Markovian models remain the most widely studied and analytically tractable class, with a mature literature on mean-field approximations (Keeling and Eames, 2005; Kiss et al., 2017) and on control-theoretic formulations (Nowzari, Preciado and Pappas, 2016; Watkins, Nowzari, Preciado and Pappas, 2019; Preciado, Zargham, Enyioha, Jadbabaie and Pappas, 2014) that exploit memoryless dynamics. Nevertheless, a practical simulation framework must cover *both* regimes efficiently, not just one.

We present FLASHSPREAD, a unified GPU framework that addresses this computational duality. Its contribution is primarily computational: a fused Triton renewal kernel that consolidates the entire per-step pipeline into a single launch while respecting CUDA Graph capture, and a degree-aware CSR dispatch that adapts the thread mapping to the graph’s degree heterogeneity. To the best of our knowledge it is also the first open-source, end-to-end GPU framework for non-Markovian (renewal) spreading on networks with kernel-fused dense stepping. Recent exact event-driven CPU methods such as NEXT-Net (Cure, Pflug and Pigoletti, 2025) already reach  $\sim 10^6$  nodes per simulation; FLASHSPREAD is complementary, extending synchronous tau-leaping to  $10^8$  on a single A100 with the per-step compute pipeline fused into a single GPU kernel. Our specific contributions are:

1. **GPU-accelerated renewal epidemics at scale.** A first open-source, end-to-end GPU framework for non-Markovian network epidemic simulation with kernel-fused dense stepping, scaling to  $N = 10^8$  nodes on a single A100 and achieving 8.09 Giga-NUPS on a uniform-degree graph at  $N = 10^6$ , a  $217\times$  strict hardware speedup over the CPU tau-leaping baseline measured on the same network.
2. **Fused renewal engine plus a companion Markovian engine.** The scientific core of this paper is the dense  $\mathcal{O}(N/P)$  fused renewal engine; the Markovian engine is a secondary, companion contribution that the framework additionally provides. We identify that the two regimes demand opposite GPU strategies — sparse  $\mathcal{O}(K \cdot D_{\text{avg}}/P)$  incremental updates for memoryless dynamics versus the renewal engine’s dense synchronous stepping — and validate both against exact references within their relevant regime (generalised non-Markovian Gillespie for SEIR in Section 6; Doob–Gillespie for SIS and SIR in Section 6.1 and Appendix C.7).
3. **IO-aware kernel fusion for dense renewal dynamics.** Building on the IO-aware design principle of Dao, Fu, Ermon, Rudra and Ré (2022), we consolidate CSR traversal, numerically stable  $\text{erfcx}$ -based hazard evaluation, adaptive Bernoulli tau-leaping, and the next-step infectivity write-back into a single fused Triton kernel. All intermediates remain in streaming-multiprocessor registers; block-scalar sparsity skips the hazard for inert thread blocks without breaking CUDA Graph capture.
4. **Degree-aware CSR dispatch.** To cope with highly heterogeneous degree distributions, we add a warp-per-node kernel and an edge-partitioned merge-based kernel (Merrill and Garland, 2016) and auto-select among the three from the graph’s  $D_{\text{max}}/D_{\text{avg}}$  ratio. The merge strategy delivers  $4.5\times$  throughput on a  $10^6$ -node Barabási–Albert graph while the auto-dispatch rule preserves peak throughput on regular graphs (Section 5.5, Appendix B).
5. **Tau-leaping fidelity against an exact reference.** We quantify the structural-bias floor of synchronous Bernoulli updates on contact networks against an exact non-Markovian Gillespie reference: the per-run peak- $I$  error is  $\sim 6\%$  and does not detectably decrease below that floor as  $\varepsilon \rightarrow 0$  across two decades of tolerance on the validation benchmarks (Appendix C).

6. **Active-node compaction for temporal sparsity.** We exploit the inter-step temporal sparsity that synchronous tau-leaping typically overlooks by shrinking the fused-kernel grid from  $\lceil N/B \rceil$  blocks to  $\lceil |X \neq R|/B \rceil$  via a Fixed-Grid Early-Exit pattern compatible with CUDA Graph capture. On saturating scale-free epidemics this delivers a **1.53×** whole-run speedup on BA  $m=4$  at  $N=10^6$  with bit-identical compartment counts (Section 5.6).
7. **Mixed-precision storage with an fp32 accumulator contract.** We downcast state (int8), age (fp16), infectivity (bf16), and weights (bf16) while holding the pressure accumulator and all kernel math in fp32. The storage-only compression delivers a **1.14×** throughput lift at  $N = 10^6$  and extends the L2-reachable scale of the fused kernel by roughly  $3\times$  (reaching  $N = 10^8$  on a single A100), within a  $\sim 0.1\%$  deviation in final attack rate, well below the structural-bias floor (Section 5.7).

## 2. Related work

The Gillespie algorithm (Gillespie, 1977, 1976) provides exact stochastic simulation of continuous-time Markov processes. The Generalized Epidemic Modeling Framework (GEMF) of Sahneh, Scoglio and Van Mieghem (2013) provides the mean-field formulation that underpins a family of compartmental simulators on multilayer networks; two high-performance simulator implementations of that family serve as our CPU baselines, c-GEMF (Sahneh, Vajdi, Shakeri, Fan and Scoglio, 2017) (C, Next Reaction Method) and FastGEMF (Samaei, Sahneh and Scoglio, 2025) (Python, vectorized updates). For non-Markovian systems, Boguñá, Lafuerza, Toral and Serrano (2014) introduced a generalized Gillespie algorithm that tracks the age  $\tau_i$  of each process. The EoN package (Miller and Ting, 2020) provides efficient non-Markovian simulation without GPU acceleration. More recently, NEXT-Net (Cure et al., 2025) reformulated the next-reaction method so that one serial CPU simulation of an exact non-Markovian process scales to networks on the order of  $10^6$  nodes, closing most of the scale gap between exact event-driven CPU simulation and approximate synchronous methods. The FLASHSPREAD contribution is therefore not a scale claim against all CPU methods: it is a scale claim under *synchronous tau-leaping* (the regime that exposes dense  $\mathcal{O}(N+E)$  parallelism) together with a first open-source, end-to-end GPU framework for renewal-process epidemics on networks with kernel-fused dense stepping, reaching  $N = 10^8$  on a single A100. Recent work has characterized when non-Markovian and Markovian dynamics yield equivalent behavior (Feng et al., 2019; Starnini, Gleeson and Boguñá, 2017), finding that equivalence generally fails for transient dynamics relevant to epidemic forecasting.

Gillespie’s tau-leaping (Gillespie, 2001) accelerates stochastic simulation by approximating event counts as Poisson random variables. Cao, Gillespie and Petzold (2006) developed adaptive step selection, and Chatterjee, Vlachos and Katsoulakis (2005) introduced binomial tau-leaping. Our *Bernoulli tau-leaping* differs fundamentally: each node undergoes at most one transition per step, naturally modeled by Bernoulli rather than Poisson statistics. This is not an approximation against the binomial formulation: on the compartmental SEIR state space each node can change compartment at most once in any infinitesimal sub-interval, and the tau-leap is selected to bound the per-step transition probability by  $\epsilon \ll 1$ ; the binomial distribution therefore reduces to Bernoulli with higher-order correction  $\mathcal{O}(\epsilon^2)$  per node, which remains below the structural-bias floor of synchronous updates (Appendix C).

GPU acceleration of epidemic simulation has primarily focused on ensemble parallelization (Perumalla, 2006; Zou, Lv, Deng and Ji, 2013) and agent-based models (Bisset, Chen, Feng, Kumar and Marathe, 2009). Komarov and D’Souza (2012) developed GPU-accelerated exact Gillespie for chemical networks, and Nobile, Cazzaniga, Besozzi and Mauri (2014) implemented GPU tau-leaping for biochemical systems. To our knowledge, no prior open-source work has published a kernel-fused, end-to-end GPU pipeline for non-Markovian (renewal) epidemic simulation on networks; the challenge is fundamental, as the sparse update strategies that make Markovian GPU simulation efficient fail when all node hazards change continuously with time. Dao et al. (2022) demonstrated that IO-aware kernel fusion (keeping intermediates in on-chip SRAM rather than writing to off-chip DRAM) can dramatically accelerate memory-bound workloads; we apply this principle to graph aggregation using a custom Triton kernel.<sup>1</sup>

Mature GPU graph frameworks such as cuGraph (NVIDIA Corporation, 2019) and Gunrock (Wang, Davidson, Pan, Wu, Riffel and Owens, 2016) do provide efficient CSR-structured SpMV and traversal primitives, and an obvious question is why not assemble the renewal step from them. We do not, for two reasons. First, neither framework

<sup>1</sup>While frameworks such as Julia/CUDA.jl or DifferentialEquations.jl offer automatic kernel fusion via broadcast graphs, they cannot accommodate the data-dependent control flow of our block-scalar skip (Section 5.4). Combined with the static operator requirements of CUDA Graph capture, achieving the optimised fusion path quantified in Figure 16 required a custom Triton kernel.

**Table 1**

Comparison with existing network epidemic simulation software.

| Package            | GPU support | Non-Markovian dynamics | Edge-w / multilayer | Kernel Fusion (IO-aware) | Max scale (nodes $N$ )   | Language   | Ref.                                  |
|--------------------|-------------|------------------------|---------------------|--------------------------|--------------------------|------------|---------------------------------------|
| GEMFsim            | ✗           | ✗                      | ✓ / ✓               | ✗                        | $10^5$                   | C          | Sahneh et al. (2017)                  |
| FastGEMF           | ✗           | ✗                      | ✓ / ✓               | ✗                        | $10^5$                   | Python     | Samaei et al. (2025)                  |
| EpiModel           | ✗           | ✗                      | ✓ / ✗               | ✗                        | $10^4$                   | R          | Jenness, Goodreau and Morris (2018)   |
| NDlib              | ✗           | ✗                      | ✗ / ✗               | ✗                        | $10^5$                   | Python     | Rossetti, Milli and Rinzivillo (2018) |
| epydemic           | ✗           | ✗                      | ✗ / ✗               | ✗                        | $10^5$                   | Python     | Dobson (2017)                         |
| EoN                | ✗           | ✓                      | ✓ / ✗               | ✗                        | $10^5$                   | Python     | Miller and Ting (2020)                |
| NEXT-Net           | ✗           | ✓                      | ✓ / ✗               | ✗                        | $10^6$                   | C++/Python | Cure et al. (2025)                    |
| <b>FlashSpread</b> | ✓           | ✓                      | ✓ / ✓               | ✓                        | <b><math>10^8</math></b> | Python     | this work                             |

**Non-Markovian dynamics:** supports age-dependent transition rates (renewal / log-normal / Weibull holding times), not only exponential (Markovian) waiting times. **Edge-weighted / multilayer:** first check indicates support for weighted edges; second check indicates support for multiple interaction layers. **Kernel Fusion:** consolidates CSR graph traversal, hazard evaluation, and stochastic sampling into a single GPU kernel launch with intermediates kept in on-chip registers (IO-aware design). **Max scale:** largest network size demonstrated in the corresponding reference; NEXT-Net (Cure et al., 2025) uses an exact next-reaction reformulation on CPU and reports runs on networks with  $\sim 10^6$  nodes, the current state of the art for exact event-driven CPU simulation. The **FLASHSPREAD  $10^8$**  entry is the approximate synchronous tau-leaping ceiling under mixed-precision storage on a single A100 (40 GB) and is therefore not directly comparable to NEXT-Net’s exact-simulation claim.

exposes a single-kernel fusion point at which stochastic sampling (random draws per node) and age-dependent  $\text{erfcx}$ -based hazard evaluation can co-reside with CSR traversal in streaming-multiprocessor registers; wrapping them would reintroduce the  $\mathcal{O}(N)$  intermediate-tensor traffic that our fused kernel removes (Appendix D.3, Figure 16). Second, our degree-aware auto-dispatch (Section 5.5) is tied to the *per-simulation-step* compute profile — state, age, infectivity, and RNG all change every step — rather than to a once-off graph query, which is the regime these libraries optimise for. We additionally rely on the CUDA Graphs API (NVIDIA Corporation, 2024) to capture the  $b$  repetitions of the fused step as a single replayable unit, eliminating per-step launch overhead (Section 5.4).

Table 1 compares FLASHSPREAD with existing packages. FLASHSPREAD demonstrates  $N = 10^8$  on a single A100 (40 GB); at  $N = 10^7$  the fp32 CSR graph structure ( $\sim 640$  MB for degree  $d = 8$ ) already exceeds the GPU’s 40 MB L2 cache, making every indirect neighbour lookup a full DRAM transaction, and the mixed-precision storage path of Section 5.7 (bf16 weights, int8 state, fp16 age, bf16 infectivity) compresses the per-step working set enough to reach  $N = 10^8$  within the 40 GB HBM budget. Scaling beyond  $N \sim 10^8$  exceeds single-GPU global memory and requires multi-GPU domain decomposition.

### 3. Problem formulation

Consider a contact network  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{W})$  with  $N = |\mathcal{V}|$  nodes and  $L$  interaction layers. Each node  $i$  occupies one of  $M$  compartments. The system state is:

$$(\mathbf{X}(t), \boldsymbol{\tau}(t)) \in \{1, \dots, M\}^N \times \mathbb{R}_{\geq 0}^N \quad (1)$$

where  $\mathbf{X}(t)$  is the compartmental state vector and  $\boldsymbol{\tau}(t)$  is the *age tensor*, the time each node has spent in its current state. The age resets to zero upon any state transition (the *renewal property*).

The instantaneous transition rate for node  $i$  from compartment  $m$  to  $n$  decomposes into nodal and edge-mediated components:

$$\lambda_i^{m \rightarrow n}(t) = \underbrace{\Psi_{\text{node}}^{m \rightarrow n}(\tau_i, \mathbf{u})}_{\text{spontaneous}} + \underbrace{\sum_{\ell=1}^L \Psi_{\text{edge}}^{m \rightarrow n, \ell}(\mathcal{I}_i^{(\ell)}, \mathbf{u})}_{\text{contact-driven}} \quad (2)$$

The *inducer influence*  $\mathcal{I}_i^{(\ell)}$  aggregates contributions from neighbors weighted by an *infectivity function*  $\rho^{(\ell)}$ :

$$\mathcal{I}_i^{(\ell)}(\mathbf{X}, \boldsymbol{\tau}) = \sum_{j \in \mathcal{N}_{\text{in}}^{(\ell)}(i)} w_{ji}^{(\ell)} \cdot \rho^{(\ell)}(X_j, \tau_j) \quad (3)$$

For standard Markovian models,  $\rho^{(\ell)}(X_j, \tau_j) = \mathbf{1}\{X_j = q^{(\ell)}\}$  is a binary indicator. For non-Markovian models with age-dependent transmission,  $\rho^{(\ell)}$  can be a continuous function of the source node's infection age  $\tau_j$  (Section 5.3), enabling biologically realistic viral shedding curves without incurring  $\mathcal{O}(E)$  per-edge memory costs.

The *computational duality* central to FLASHSPREAD arises from how the nodal rate depends on age. In the Markovian case, the rate is age-independent, enabling  $\mathcal{O}(K \cdot D_{\text{avg}})$  sparse updates. In the renewal case, the rate depends on holding time via the hazard function  $h(\tau) = f(\tau)/S(\tau)$ , requiring dense  $\mathcal{O}(N)$  updates at every step.

## 4. Computational engines

FLASHSPREAD provides two engines (Markovian, Renewal) and three CSR traversal strategies (thread, warp, merge) that are typically abstracted from the user: a compact decision tree, reproduced in Figure 3, drives the runtime dispatch. The remainder of this section outlines the engine-level mechanics referenced in Figure 3; Sections 5 and 5.5 cover the renewal side. Both engines share the FlashNeighbor kernel for computing inducer influence (Eq. 3). A naive approach requires materializing dense indicator vectors, incurring  $2NL$  memory transactions for discarded intermediates. We fuse operations into a single kernel that streams CSR edges, looks up neighbor states via indirect memory access, evaluates predicates in registers, and accumulates weighted sums, writing only the final influence tensor to global memory. The graph is stored in CSR format indexed by *incoming* edges, enabling *gather-based parallelism*: each GPU thread owns a unique target node and accumulates influence in private registers without atomic operations. The concrete data layout, per-thread register usage, and the corresponding traffic through HBM / L2 / SM registers are documented in Appendix D (Figures 14–16); these details drive the  $\sim 64N \rightarrow \sim 20N$  bytes/step fusion gain reported in Section 5.4.

Listing 1 shows the user-facing Python API, which is deliberately kept flat so that the Markovian / renewal engine choice, the CSR traversal strategy, and the tau-leaping tolerance are all explicit in the engine constructor.

For Markovian dynamics, transition rates are piecewise constant, enabling two operational modes: *Control Mode* performs dense FlashNeighbor passes when control inputs change ( $\mathcal{O}((N+E)/P)$ ), while *Inertial Mode* exploits event-driven sparsity with  $\mathcal{O}(|\mathcal{T}| \cdot D_{\text{avg}}/P)$  updates where  $|\mathcal{T}| \ll N$ . To bound accumulated rounding from the repeated sparse atomic updates, Inertial Mode periodically triggers a full Control-Mode recomputation (every 200 events in our implementation); this cadence is a tunable accuracy knob rather than a correctness requirement. Algorithm 1 summarizes the step logic. This sparse engine achieves  $> 2 \times 10^7$  events/sec at  $N = 10^6$ , but its optimization strategy (incremental updates between rare events) is inapplicable to renewal processes where all hazards change continuously.

---

### Algorithm 1 Markovian Engine: One Tau-Leaping Step

---

**Require:** State  $\mathbf{X}$ , rates  $\lambda$ , influence  $\mathbf{I}$ , parameters  $\theta, p_{\text{max}}, \tau_{\text{max}}$

**Ensure:** Updated state  $\mathbf{X}'$ , elapsed time  $\tau$

```

1:  $\Lambda \leftarrow \sum_i \lambda_i$  ▷ Total rate
2:  $\tau \leftarrow \min\left(\frac{\theta \cdot N}{\Lambda}, \frac{p_{\text{max}}}{\max_i \lambda_i}, \tau_{\text{max}}\right)$ 
3:  $p_i \leftarrow 1 - e^{-\lambda_i \tau}$  for all  $i$  ▷ Transition probabilities
4:  $\mathbf{m} \leftarrow \text{Bernoulli}(\mathbf{p})$  ▷ Sample event mask
5:  $\mathbf{X}' \leftarrow \text{APPLYTRANSITIONS}(\mathbf{X}, \mathbf{m})$ 
6: if few transitions (Inertial Mode) then
7:   Update  $\mathbf{I}$  via sparse atomics on outgoing CSR
8: else
9:    $\mathbf{I} \leftarrow \text{FLASHNEIGHBOR}(\mathbf{X}')$  ▷ Control Mode
10: end if
11:  $\lambda \leftarrow \text{COMPUTERATES}(\mathbf{X}', \mathbf{I})$ 
```

---

## 5. The Renewal engine

For renewal processes, all  $N$  hazards change at every time step, precluding the sparse updates that make Markovian simulation efficient. This section presents our core contribution: a fused GPU kernel that achieves memory-bandwidth-limited execution for dense renewal dynamics.



### 5.1. Numerically stable log-normal hazards

Direct computation of  $h(\tau) = f(\tau)/S(\tau)$  suffers from catastrophic cancellation for large  $\tau$ . We derive a stable formulation using the scaled complementary error function:

**Proposition 1** (Stable Log-Normal Hazard). *For a log-normal distribution with parameters  $(\mu, \sigma)$ :*

$$h_{\text{LN}}(\tau; \mu, \sigma) = \sqrt{\frac{2}{\pi}} \frac{1}{\tau \sigma \operatorname{erfcx}(z)}, \quad z = \frac{\ln \tau - \mu}{\sigma \sqrt{2}} \quad (4)$$

*Proof.* The log-normal density and survival function are  $f(\tau) = \frac{1}{\tau \sigma \sqrt{2\pi}} e^{-z^2}$  and  $S(\tau) = \frac{1}{2} \operatorname{erfc}(z)$ . The hazard is:

$$h(\tau) = \frac{f(\tau)}{S(\tau)} = \sqrt{\frac{2}{\pi}} \cdot \frac{1}{\tau \sigma} \cdot \frac{1}{e^{z^2} \operatorname{erfc}(z)} = \sqrt{\frac{2}{\pi}} \cdot \frac{1}{\tau \sigma \cdot \operatorname{erfcx}(z)} \quad (5)$$

where  $\operatorname{erfcx}(z) = e^{z^2} \operatorname{erfc}(z)$  remains well-conditioned for all  $\tau > 0$  via `torch.special.erfcx`.  $\square$

### 5.2. Bernoulli tau-leaping

In compartmental epidemic models, each node can undergo *at most one* transition per step. The appropriate model is Bernoulli:

$$\text{transition}_i \sim \text{Bernoulli}(p_i), \quad p_i = 1 - e^{-\lambda_i \Delta t} \quad (6)$$

with adaptive step selection:

$$\Delta t = \min \left( \Delta t_{\max}, \frac{\varepsilon}{\max_i \lambda_i + \delta} \right) \quad (7)$$

where  $\varepsilon \in [0.01, 0.05]$  bounds the maximum transition probability per step. As  $\varepsilon \rightarrow 0$ , the approximation converges to the exact continuous-time dynamics; our validation (Section 6) demonstrates that  $\varepsilon = 0.03$  produces trajectories in close agreement with exact Gillespie methods.

Achieving GPU acceleration necessitates a fundamental algorithmic paradigm shift. In existing sequential non-Markovian simulators, the continuous re-evaluation of hazard rates relies on a vast system size ( $N \gg 1$ ) to naturally produce infinitesimal time steps (Boguñá et al., 2014). By employing a first-order expansion, this approach inherently imposes an approximation that mathematically freezes the hazard rates during each interval. While highly accurate in the thermodynamic limit, this finite- $N$  approximation degrades if intrinsic system rates drop and the natural time step becomes too large. To guarantee tight bounds and prevent this degradation, sequential models often inject an independent auxiliary Markovian process, effectively acting as a phantom process (Vajdi, 2020). This approach floods the simulation with  $\mathcal{O}(\lambda_0)$  dummy events per unit time. Because the auxiliary rate  $\lambda_0$  must be kept aggressively large to restrict integration error, the strict sequential event queue becomes an insurmountable bottleneck, choked by zero-state-change events. Consequently, tau-leaping is not merely a performance heuristic; it is the required algorithmic paradigm to break this sequential dependency and expose the dense  $\mathcal{O}(N)$  data parallelism that the GPU subsequently exploits.

### 5.3. Source-node approximation for age-dependent edges

Exact non-Markovian simulation typically tracks per-edge ages at  $\mathcal{O}(E \times M)$  memory cost to enable arbitrary edge dynamics (Boguñá et al., 2014). However, we can bypass this massive memory footprint by adopting a host-centric approach—theoretically equivalent to “Rule 2” in Boguñá et al. (2014). By observing that viral shedding is fundamentally a property of the *host* rather than the *contact*, we define the instantaneous infectivity as:

$$\rho^{(\ell)}(X_j, \tau_j) = \beta \cdot s(\tau_j) \cdot \mathbf{1}\{X_j = q^{(\ell)}\} \quad (8)$$

where  $s(\tau_j)$  is a user-defined viral shedding profile reflecting the time course of infectiousness (e.g., a log-normal density calibrated to empirical viral load data (He et al., 2020)). This formulation decouples a node’s external infectiousness, governed by  $s(\tau)$ , from its internal recovery dynamics governed by the hazard  $h(\tau)$ . Setting  $s(\tau_j) = 1$  recovers standard Markovian edge semantics.<sup>2</sup>

<sup>2</sup>The approximation is exact when the per-contact transmission rate depends only on the source node’s infection age, which is the standard epidemiological assumption for viral shedding profiles. It breaks down in three families of scenarios: (i) *dose-dependent transmission*, where the

#### 5.4. Fused kernel with block-level sparsity

The unfused renewal pipeline executes  $\sim 12$  separate kernel launches per step, materializing  $\sim 7$  intermediate  $\mathcal{O}(N)$  buffers to global memory (pressure, rates, event probabilities, random samples, event masks, next state, age updates), totaling  $\sim 64N$  bytes of traffic. We consolidate everything into a single fused kernel, reducing traffic to  $\sim 20N$  bytes per step. A fundamental tension arises between compute sparsity and CUDA Graph compatibility. During a typical SEIR epidemic,  $\sim 70\%$  of nodes occupy states S or R where the expensive `erfcx`-based hazard ( $\sim 55$  FLOPs) need not be evaluated. In PyTorch, exploiting this requires data-dependent guards that force CPU–GPU synchronization and break CUDA Graph capture. Our fused kernel resolves this by performing a block-scalar reduction over a per-block E/I-existence flag that produces a true hardware branch rather than per-lane predication: thread blocks containing no active nodes skip the entire hazard evaluation, while blocks with at least one E or I node evaluate the hazard and mask-select the result across lanes. This preserves CUDA Graph capture while still recovering most of the compute sparsity.

The GPU kernel framework lacks a native `erfcx` instruction, necessitating a piecewise identity-based (for  $|z| \leq 3.5$ ) and asymptotic (for  $|z| > 3.5$ ) approximation inside the kernel. The measured maximum relative error of this approach is  $\sim 4 \times 10^{-2}$  at the branch-switch  $z \approx 3.5$  and  $\sim 6 \times 10^{-3}$  elsewhere on an fp32-safe grid (Appendix A). Both errors are well within the tolerance of stochastic tau-leaping, as the induced bias in the Bernoulli probability scales as  $\lambda_i \Delta t \lesssim \varepsilon$  and stays far below the  $\sim 6$ – $7\%$  structural-bias floor of synchronous updates (Section 6;  $\sim 6\%$  on peak- $I$ ,  $\sim 7\%$  on final- $R$ ).

---

**Algorithm 2** Fused Renewal Engine with CUDA Graph: Capture

---

**Require:** CSR graph  $(R, C, W)$ , SEIR model parameters  $(\beta, \mu_{EI}, \sigma_{EI}, \mu_{IR}, \sigma_{IR})$ , batch size  $b$ , RNG seed  $s$ , tolerance  $\varepsilon$ , cap  $\tau_{\max}$

- 1: **Persistent state:** tensors  $\mathbf{X}, \boldsymbol{\tau}$ , scalars  $\tau_{\text{prev}} \in \mathbb{R}_{\geq 0}$ ,  $\text{step\_id} \in \mathbb{N}$
- 2: Initialise  $\tau_{\text{prev}} \leftarrow \tau_{\max}$  and  $\text{step\_id} \leftarrow 0$
- 3: Snapshot all tensor state
- 4: Warmup: execute 3 steps for JIT compilation
- 5: Capture  $b$  repetitions of FUSEDSTEP (Alg. 3) as CUDA Graph  $\mathcal{G}$
- 6: Restore tensor state from snapshot
- 7: **return**  $\mathcal{G}$ , initialised persistent state

---

Figure 4 diagrams the per-thread control flow inside Algorithm 3. Each thread opens with five coalesced HBM reads (`state`, `age`, `pressure`, `infectivity`, RNG state), then participates in two block-scalar reductions (`any_E`, `any_I`) that produce a *block-level* branch into either a skip path (no `erfcx` FLOPs) or an evaluate path (all lanes compute the log-normal hazard and mask-select the result). The Bernoulli sample, state transition, renewal age reset, and next-step infectivity write all happen in registers, and the thread closes with a single coalesced HBM write that commits the updated `state`, `age`, and next-step `infectivity`. No thread writes intermediate tensors to HBM — the essential property that makes the fused design compatible with CUDA Graph capture of many repetitions and that keeps the arithmetic intensity on the memory-bound side of the roofline (Table 8).

#### 5.5. CSR traversal strategies and auto-dispatch

The fused kernel described so far assigns one thread per node and iterates that node’s incoming neighbors in a scalar loop. This is optimal when the degree distribution is narrow (regular, Erdős–Rényi, or lightly perturbed lattices), because the loop count across a 128-thread block is uniform and memory accesses coalesce. On heavy-tailed topologies (scale-free networks, social graphs), a single hub can dominate the block’s runtime while the other 127 threads wait: measured throughput on a Barabási–Albert graph drops by more than an order of magnitude despite equivalent  $N$  and average degree (Section 6). We therefore provide two additional traversal kernels and let the engine select among them at construction time:

---

per-edge contribution depends on the cumulative contact history between a specific (source, target) pair rather than only on the source’s shedding trajectory; (ii) *temporal networks with independently forming edges*, where each edge itself has a lifetime that must be tracked; and (iii) *heterogeneous per-edge susceptibility* drawn from a non-trivial distribution, when the goal is to replicate that exact distributional shape rather than an ensemble-mean effect. In all three cases true per-edge age tracking at  $\mathcal{O}(E)$  memory is required; we view this as an extension of the renewal engine rather than a correction to the present paper.



**Algorithm 3** Fused Renewal Engine with CUDA Graph: per-call replay of  $b$  fused steps (FUSEDSTEP). Note:  $\tau_{\text{prev}}$  is initialised to  $\tau_{\text{max}}$  in Algorithm 2, so the *first* step of each replay uses  $\tau_{\text{max}}$  as a conservative upper bound on the step size; subsequent steps use the adaptive  $\tau$  written by the previous iteration's tau-update line. This introduces at most one over-conservative step per  $b=50$ -step replay window and is the direct cost of keeping the adaptive step inside a capturable graph — the alternative, recomputing  $\tau$  outside the graph every step, would eliminate the gain of CUDA Graph batching.

**Require:** State  $\mathbf{X}$ , age  $\tau$ , CSR graph  $(R, C, W)$ , model parameters  $(\beta, \mu, \sigma)$ , batch size  $b$ , seed  $s$ ; captured graph  $\mathcal{G}$

**Ensure:** Updated  $(\mathbf{X}', \tau')$ , elapsed time  $T$

```

1: Persistent state (carried across calls):  $\tau_{\text{prev}}, \text{step\_id}$ ; initialised by Alg. 2
2:  $T \leftarrow 0$ 
3:  $\mathcal{G}.\text{replay}()$  ▷ Executes  $b$  fused steps:
4: for  $k = 1$  to  $b$  do ▷ (inside captured graph)
5:    $T \leftarrow T + \tau_{\text{prev}}$ 
6:   ▷ Step 1: Infectivity pre-pass ( $\mathcal{O}(N)$ , elementwise)
7:   infectivity[ $j$ ]  $\leftarrow \beta \cdot s(\tau_j) \cdot \mathbf{1}\{X_j = I\}$  for all  $j$ 
8:   step_id  $\leftarrow \text{step\_id} + 1$ 
9:
10:  ▷ Step 2: Fused GPU kernel (single launch, per thread  $i$ ):
11:   $p_i \leftarrow 0$  ▷ CSR traversal  $\rightarrow$  pressure in registers
12:  for  $j \in \mathcal{N}_{\text{in}}(i)$  via CSR do
13:     $p_i \leftarrow p_i + \text{infectivity}[j] \cdot W[j]$ 
14:  end for
15:  Load  $X_i, \tau_i$  from global memory
16:   $\lambda_i \leftarrow p_i$  if  $X_i = S$ ;  $\lambda_i \leftarrow 0$  if  $X_i = R$ 
17:  if any  $E$ -node in block then
18:     $\lambda_i \leftarrow h_{\text{LN}}(\tau_i; \mu_{EI}, \sigma_{EI})$  for  $E$ -nodes ▷ block-level skip
19:  end if
20:  if any  $I$ -node in block then
21:     $\lambda_i \leftarrow h_{\text{LN}}(\tau_i; \mu_{IR}, \sigma_{IR})$  for  $I$ -nodes
22:  end if
23:   $q_i \leftarrow 1 - e^{-\lambda_i \cdot \tau_{\text{prev}}}$ ;  $u \leftarrow \text{RAND}(s + \text{step\_id}, i)$ 
24:   $X'_i \leftarrow \text{TRANSITION}(X_i, u < q_i)$ 
25:   $\tau'_i \leftarrow 0$  if  $X'_i \neq X_i$ , else  $\tau_i + \tau_{\text{prev}}$  ▷ renewal reset
26:  Store  $X'_i, \tau'_i, \lambda_i$  ▷ single global write
27:
28:  ▷ Step 3: Tau update ( $\mathcal{O}(N)$  reduction)
29:   $\tau_{\text{prev}} \leftarrow \min(\tau_{\text{max}}, \varepsilon / (\max_i \lambda_i + \delta))$ 
30: end for

```

- *Warp-per-node.* The block is laid out as  $[\text{NODES\_PER\_BLOCK}, \text{LANES\_PER\_NODE}]$  with  $\text{LANES\_PER\_NODE} = 32$ . Each warp cooperates on one node's neighbor list, iterating  $\lceil D_i/32 \rceil$  chunks in parallel, then a warp-local reduction yields the node's pressure. This reduces hub traversal from  $\mathcal{O}(D_{\text{max}})$  to  $\mathcal{O}(D_{\text{max}}/32)$  but still leaves inter-warp idle time inside a block that mixes one hub with several low-degree nodes.
- *Edge-partitioned merge-based.* Following the classic merge-path load-balancing idea of Merrill and Garland (2016), we launch one program per fixed chunk of  $\text{EDGES\_PER\_BLOCK}$  contiguous edges. Each lane loads one edge's (col\_ind, weight, infectivity[neighbor]), recovers the source node id via a fully unrolled binary search of depth  $\lceil \log_2(N + 2) \rceil$  on row\_ptr, and atomic-adds the weighted contribution to a shared pressure buffer. A cheap second kernel then runs the per-node tail (hazard, Bernoulli, transition, next-infectivity write) exactly as in Algorithm 3. Load balance across blocks is perfect regardless of degree skew, but the scheme costs one atomic per edge, one scratch buffer read/write, and one extra kernel launch.

**Table 2**

CSR traversal strategy throughput (Fused CG,  $b = 50$ ,  $N = 10^6$ , mean degree 8). The regular graph has  $\rho = D_{\max}/D_{\text{avg}} = 2$  and auto-dispatch picks `thread`; the BA graph has  $\rho = 484$  and auto-dispatch picks `merge`. Each strategy produces statistically equivalent trajectories; only wall-clock differs.

| Strategy                              | Regular ( $d = 8$ )<br>G-NUPS | BA ( $m = 4$ )<br>G-NUPS | BA / Regular<br>% |
|---------------------------------------|-------------------------------|--------------------------|-------------------|
| <code>thread</code> (1 thread / node) | 7.88                          | 0.45                     | 5.7%              |
| <code>warp</code> (32 threads / node) | 1.70                          | 1.30                     | 76.5%             |
| <code>merge</code> (edge-partitioned) | 3.92                          | 2.00                     | 51.0%             |

The three strategies are bit-exact equivalent (to within floating-point reduction order) because they all use the same counter-based RNG seeded by global node id and step counter, and the same per-node tail logic. We verified this on  $N = 10^4$  regular and BA graphs: `thread` and `warp` are bit-identical every step; `merge` uses non-deterministic atomic-add ordering and therefore matches only at population-count granularity (zero-delta in our tests).

*Auto-dispatch.* At engine construction the framework inspects the graph’s `row_ptr` once, computes  $\rho = D_{\max}/D_{\text{avg}}$ , and picks:

$$\text{strategy} = \begin{cases} \text{thread} & \rho < \rho_w, \\ \text{warp} & \rho_w \leq \rho < \rho_m, \\ \text{merge} & \rho \geq \rho_m, \end{cases}$$

with  $(\rho_w, \rho_m) = (4, 50)$  calibrated against the benchmarks in Table 2. The user can override the choice via an explicit strategy flag at engine construction. The dispatch cost (one pass over `row_ptr`) is amortized over every subsequent simulation step.

Relative to the default 1-thread-per-node kernel, the merge-based kernel is 4.5× faster on BA but 2× slower on a regular graph, which is exactly why the auto-dispatch heuristic is needed: the same kernel cannot be optimal for both regimes, and the user should not have to know which one their graph falls into. For each strategy, `NODES_PER_BLOCK` (`warp`) and `EDGES_PER_BLOCK` (`merge`) are small tunables with modest impact (Section 6).

## 5.6. Epidemic-aware compute sparsity: active-node compaction

The fused kernel of Section 5.4 already skips the `erfcx` hazard for thread blocks that contain no *E* or *I* node; the block-scalar reduction harvests *intra-step* compute sparsity along the spatial dimension (which blocks are inert this step). A natural complementary optimisation harvests *inter-step temporal* sparsity: once a node reaches the absorbing state *R*, it takes no further action, and every replay step it still participates in (a block it belongs to, an  $\mathcal{O}(N)$  copy-back of `state/age/infectivity`) is pure overhead. Dropping recovered nodes from the per-step workload shrinks the kernel’s block count from  $\lceil N/B \rceil$  toward  $\lceil |X \neq R|/B \rceil$ , which approaches  $\mathcal{O}(1)$  as the tail of a saturating epidemic completes.

We implement this as a CUDA-Graph-compatible *active-node compaction* path. The predicate is  $X \neq R$  — not  $X \in \{E, I\}$ , because the pull-based CSR gather of Section 5.4 also requires *S* nodes to evaluate their incoming pressure in order to transition. Dropping *S* would freeze the epidemic. Because *R* is absorbing, the set of non-*R* nodes shrinks monotonically, so refreshing the active list at the CUDA Graph replay boundary (once per  $b = 50$ -step window) stays correct even when nodes transition to *R* mid-replay: they remain on the list doing harmless `rate = 0` evaluations until the next refresh.

*Fixed-grid, early-exit pattern.* To keep the refresh outside the captured CUDA Graph while the kernel launch inside stays immutable, we pre-allocate two static buffers: an `int32 active_nodes[N+B]` holding the sorted non-*R* node ids (zero-padded past *N* to make any tail-block masked load well-defined) and a scalar `int32 num_active`. The captured kernel grid is fixed at  $\lceil N/B \rceil$ . On entry, the kernel loads `num_active`, sets `mask = offsets < num_active`, and remaps `idx = active_nodes[offsets]` through a masked gather; inactive tail lanes produce `idx = 0` but contribute nothing because every subsequent `tl.load/tl.store` is mask-gated. The refresh itself is a single `torch.nonzero + in-place copy_` between replays; the underlying pointers never move, so no

**Table 3**

Active-node compaction summary at  $N=10^6$ ,  $TF=50$ , 5 replicate trials. “Final  $R/N$ ” is the mean attack rate at  $t=50$ . NUPS is the whole-run mean. Std across replicates is below 1% in every cell. Bit-identity of compaction and baseline compartment counts was verified at five checkpoints ( $t \in \{1, 5, 15, 30, 50\}$ ) across three seeds on a paired  $N=10^4$  run.

| Graph    | Final $R/N$ | Baseline (G-NUPS) | Compaction (G-NUPS) | Speedup      |
|----------|-------------|-------------------|---------------------|--------------|
| ER $d=8$ | 0.156       | 7.60              | 7.52                | 0.99×        |
| BA $m=4$ | 0.978       | 0.44              | 0.67                | <b>1.53×</b> |

graph recapture is triggered. The only correctness subtlety is that inactive positions of the `rates` buffer retain stale, historically-nonzero values from when the corresponding nodes were last E or I; we zero `rates` once before each replay, matching the baseline invariant that `rate[R] = 0` every step. With that zeroing, the compaction path produces bit-identical compartment counts against the baseline at every step checkpoint on an  $N = 10^4$  sanity sweep (five seeded checkpoints, three independent seeds).

*Topology-dependent gain.* Figure 5 overlays NUPS( $t$ ) for both topologies on the renewal benchmark, bucketed into ten temporal windows of the  $TF = 50$  run. Table 3 gives the whole-run means. On ER  $d=8$  the epidemic saturates slowly under the benchmark parameters (final attack rate  $\approx 15\%$ ), so most of the run has `num_active`  $\approx N$  and the refresh overhead exceeds the marginal block-skip gain: compaction is  $\sim 1\%$  slower on the whole-run mean, with the per-bucket ratio crossing back above 1 only in the last temporal window. On BA  $m=4$  at the same  $N$  and  $\beta$ , explosive hub-driven takeoff reaches  $\approx 97\%$  attack rate by  $TF = 50$ , so the shrinking active set dominates almost the entire run and the per-bucket speedup climbs from 1.0× (takeoff) to 4.8× (last window, `num_active`  $\approx 3\%N$ ) for a whole-run mean of **1.53×** (Table 3). This makes compaction a high-saturation / long-horizon optimisation: it is most valuable on topologies and parameter regimes that drive the epidemic through a long tail phase (scale-free contact networks, policy evaluations that simulate past the peak, RL rollouts that deliberately run to extinction).

The compaction path is a user-opt-in feature rather than the default, and is currently wired only into the thread-traversal kernel. On scale-free graphs the auto-dispatch ordinarily selects the merge traversal, in which case compaction offers no benefit because the pressure kernel is edge-partitioned; the hybrid “thread-kernel compaction on BA” measurement in Table 3 is a best-case *forced* configuration that illustrates the upper bound of the mechanism on BA topology, not the production configuration.

### 5.7. Mixed-precision storage with an fp32 accumulator

A second lever on effective bandwidth is to store the non-accumulator state in narrower floating- and integer-point types. Table 4 lists the per-field choice. Compartmental state is a 4-state categorical variable (S, E, I, R) and fits trivially in `int8`; ages never exceed the simulation horizon of tens of days and a 10-bit fp16 mantissa preserves them to  $\sim 10^{-3}$  days; the per-node infectivity and per-edge weight tensors use `bfloat16` so the 8-bit exponent preserves the heavy-tailed lognormal shedding profile without the silent underflow fp16 would cause near the late-stage  $\beta \cdot s(\tau)$  tail. CSR indices stay `int32` to safely address  $N, E \geq 10^7$ .

*Kernel contract: promote on load, cast on store.* The thread-path fused kernel carries a `MIXED_PRECISION` constexpr flag. Under `MIXED_PRECISION=1`, every `tl.load` of `state`, `age`, `infectivity`, or `weights` is immediately cast to its natural math type (`int32` for `state`, `fp32` for the other three) using `.to(tl.float32)`; all hazard, Bernoulli, and age-update math then runs in `fp32/int32` exactly as in the baseline; only the final three write-once stores at the kernel tail cast back to `int8/fp16/bf16`. Under `MIXED_PRECISION=0` the casts compile out and the kernel is byte-identical to the pre-mixed-precision code path. This pattern eliminates the usual correctness risk of “blanket precision reduction” implementations that let reduced-precision values enter multi-step arithmetic: the only effect of the mixed storage is on HBM traffic, not on the numerical behaviour inside registers.

*Measured effect.* Table 5 reports throughput at  $N = 10^6$ ,  $TF = 50$ , 5 replicate trials per cell, on the thread-traversal kernel (the mixed-precision path is currently wired only into the thread-path fused kernel). The isolated-weights downcast path (bf16 weights only) that has been included with FlashSpread since v1.0 gives no measurable throughput change on the fused CUDA-Graph engine (the existing ablation in `results/ablation_nonmarkov/` has `fused_cg=0.118` ms/step and `fused_cg_bf16=0.118` ms/step); weight-band traffic is only  $\approx 20\%$  of total

**Table 4**

Per-tensor storage dtype in the mixed-precision path (engine flag `use_mixed_precision`). The accumulator for the pressure gather ( $p_i = \sum_j w_{ji} \cdot \text{infectivity}_j$ ) stays `float32` inside the kernel: summing hundreds of `bf16` contributions on a scale-free hub would otherwise absorb small values via mantissa underflow in the exact regime where  $\lambda_i \Delta t \ll 1$  matters most. All kernel math (hazard evaluation, Bernoulli sampling, age update) is also in `float32`; down-conversion happens only at the final store.

| Field         | Baseline | Mixed       | Rationale                                                                                |
|---------------|----------|-------------|------------------------------------------------------------------------------------------|
| state         | int32    | int8        | 4-state categorical; int8 is trivially safe.                                             |
| age           | fp32     | fp16        | 10-bit mantissa preserves $\sim 10^{-3}$ days on a $[0, 50]$ day grid.                   |
| infectivity   | fp32     | bf16        | bf16 8-bit exponent preserves the lognormal tail; fp16 would underflow.                  |
| weights       | fp32     | bf16        | Same argument as infectivity; pulled in via the existing <code>bf16_weights</code> path. |
| col_ind       | int32    | int32       | Unchanged; must safely address $N, E \geq 10^7$ .                                        |
| row_ptr       | int32    | int32       | Unchanged.                                                                               |
| pressure acc. | fp32     | <b>fp32</b> | <i>Mandatory</i> : bf16 accumulation over a 1000-edge hub absorbs small values.          |
| rates, tau    | fp32     | fp32        | Tau reduction stays fp32 so the adaptive step size is unaffected.                        |

**Table 5**

Mixed-precision throughput ablation at  $N=10^6$ ,  $\text{TF}=50$ , 5 replicate trials per cell, on the thread-traversal kernel. Fidelity columns give the mean  $|R_{\text{mixed}} - R_{\text{base}}|/R_{\text{base}}$  across paired seed-matched trials at  $N=10^4$ . <sup>‡</sup> The  $N=10^7$  “post-L2-cliff” row is measured on 3 replicate trials rather than 5 (simulations at  $N=10^7$  cost minutes per replicate; the drop keeps the bench inside a 45-min SLURM budget). Coefficient of variation across the three trials is below 4%, consistent with the sub-2% CoV observed at smaller  $N$ ; the  $R$ -error column is reported only where a paired  $N=10^4$  controlled-RNG run was available.

| Graph ( $N$ )                                   | csr_strategy    | Baseline (G-NUPS) | Mixed (G-NUPS)    | Speedup      | $R$ error |
|-------------------------------------------------|-----------------|-------------------|-------------------|--------------|-----------|
| ER $d=8$ ( $10^6$ )                             | thread          | $7.15 \pm 0.36$   | $8.16 \pm 0.68$   | 1.14×        | 0.05%     |
| ER $d=8$ ( $10^7$ , post-L2-cliff) <sup>‡</sup> | thread          | 1.33              | 3.10              | <b>2.32×</b> | —         |
| BA $m=4$ ( $10^6$ )                             | thread (forced) | $0.434 \pm 0.001$ | $0.458 \pm 0.000$ | 1.06×        | 0.00%     |
| BA $m=4$ ( $10^6$ )                             | merge (auto)    | $1.810 \pm 0.025$ | $1.829 \pm 0.024$ | 1.01×        | 0.35%     |

kernel traffic after fusion, and cutting it in half leaves the rest unchanged. The combined four-tensor downcast of Table 4 is different: it removes  $\sim 28\%$  of the remaining state/age/infectivity traffic on top of the weights saving, and the measured throughput improves by 1.141× on ER  $d=8$  and 1.056× on BA  $m=4$ . Fidelity versus the baseline fp32 path is well inside the structural-bias floor of Appendix C: on a paired  $N = 10^4$  run across three seeds the relative error in final attack rate is 0.00%–0.08%, and on the  $N = 10^6$  production runs the compartment counts at  $t = 50$  differ by  $\lesssim 70$  nodes out of  $10^6$  between matched trials. This confirms empirically what the kernel contract asserts: mixed precision is a pure storage-bandwidth optimisation whose numerical effect is bounded by bf16 quantisation of the Bernoulli threshold  $1 - \exp(-\lambda_i \Delta t)$ , which is well below the synchronous-updates bias floor.

The gain on the `thread` kernel is 1.14× on ER at  $N = 10^6$  (first row of Table 5) and 1.06× on BA at the same  $N$ ; the smaller BA-thread number reflects that the thread kernel on a scale-free graph is dominated by hub-driven warp divergence during CSR traversal rather than by state/age/infectivity bandwidth. At  $N = 10^7$ , where the fp32 CSR + state working set exceeds the A100’s 40 MB L2 capacity and all fused-kernel variants drop into the “L2 cliff” regime (Appendix D.3), mixed-precision storage delivers **2.32×** throughput (Figure 6, second row of Table 5): the narrower state/age/infectivity dtypes shrink the working set by roughly 3×, pushing the effective cliff out by the same factor without paying any compute-side cost. This is where the technique turns from a modest bandwidth improvement into a meaningful extension of the hardware-tractable scale. The production BA path does not use the thread kernel: auto-dispatch selects the merge kernel (Section 5.5), and we extend the `MIXED_PRECISION` constexpr into its tail kernel as well. Critically, the shared merge *scratch pressure buffer* that accumulates per-edge contributions via `atomicAdd` is

held at fp32 on both paths, because summing hundreds of bf16 partial pressures on a scale-free hub would otherwise absorb small values through mantissa underflow exactly in the  $\lambda_i \Delta t \ll 1$  regime where the Bernoulli threshold is most sensitive. Under this contract the merge-path mixed-precision measurement at  $N = 10^6$  on BA comes in at  $1.011\times$  baseline (third row of Table 5): essentially a null result, which is consistent with the merge kernel being *atomic-bound* on BA hubs rather than bandwidth-bound, so cutting the byte width of per-edge infectivity and weight cannot move the needle once the same `atomicAdd` stream serialises on the same destination addresses. Final compartment counts on the merge path still track the fp32 baseline within  $\lesssim 1\%$  in final attack rate (the baseline merge path is itself not bit-reproducible because `atomicAdd` ordering on the hub is non-deterministic; the structural-bias floor of Appendix C is  $\sim 6\text{--}7\%$ , a full order of magnitude larger than either the merge non-determinism or the mixed-precision quantisation). Combined with the active-node compaction of Section 5.6, the two levers are orthogonal and both fully covered by auto-dispatch: compaction shrinks the block count in the late, high-saturation phase of the epidemic, mixed precision shrinks the bytes-per-block when bandwidth is the limiter. On BA at production scale the dominant limiter on the merge path is atomic contention, not bandwidth, which motivates the segmented-reduction future work discussed in Section 7. Both flags are user-opt-in on the fused CUDA-Graph engine under the thread and merge traversals (the warp variant remains fp32-only for now).

### 5.8. Evaluated but omitted locality optimisations

This subsection documents one pre-registered locality optimisation that we measured and rejected. Documenting this negative result, the decision boundary, and the associated amortisation argument is intentional; it prevents duplicate effort in follow-up work and mitigates the well-documented publication bias against negative results in HPC benchmarking.

*Reverse Cuthill–McKee (RCM) reordering on scale-free graphs.* A second locality optimisation often applied to sparse CSR workloads is to permute node ids so that rows with nearby indices have spatially nearby neighbour lists, reducing L2 capacity misses on the indirect gather. The classic Reverse Cuthill–McKee (RCM) ordering is known to be highly effective on FEM-style banded matrices but is mismatched to the access pattern of hub-dominated scale-free graphs, where a small number of high-degree nodes touch memory across the entire `col_ind` array regardless of permutation. To quantify this on our workload, we ran the same BA  $m=4$ ,  $N = 10^6$  benchmark as Section 6 with and without RCM reordering applied to the incoming CSR. RCM yields a  $1.035\times$  throughput lift ( $1.876 \rightarrow 1.942$  G-NUPS on the production auto-dispatch path, which selects the merge kernel), well below the  $1.05\times$  threshold we pre-registered as the “RCM-sufficient” decision boundary. Furthermore, the RCM CPU preprocess itself took 593 s on the reference node, which at  $\approx 1.3$  s per GPU trial would require roughly 450 Monte Carlo trials just to amortise the preprocess cost back to break-even. For the typical ensemble sizes used in epidemic forecasting ( $10^2\text{--}10^3$  trajectories), RCM is a net regression rather than a win. This rules *out* plain bandwidth-minimisation reorderings as a useful locality lever for the fused renewal engine on scale-free graphs, and motivates future integration of a cache-line-aware reorder (Rabbit Order (Arai, Shiokawa, Yamamuro, Onizuka and Iwamura, 2016), Gorder (Wei, Yu, Lu and Lin, 2016)) whose locality objective is matched to the block-based traversal pattern of our merge kernel; we did not attempt that integration for this paper because the required C++ dependency is outside the scope of a revision that already establishes the headline mechanism, and we flag it as explicit future work in Section 7.

## 6. Experimental evaluation

All experiments were conducted on an NVIDIA A100-SXM4-80GB GPU with an AMD EPYC 7763 CPU (8 cores), using PyTorch 2.5.1, Triton 3.1, CUDA 12.1, and Python 3.11. CPU baselines include c-GEMF (Sahneh et al., 2017) (C, exact event-driven), FastGEMF (Samaei et al., 2025) (Python, exact event-driven), and our exact non-Markovian Phantom-Process implementation (Vajdi, 2020). Networks are Erdős–Rényi random graphs with average degree  $d = 8$  unless otherwise noted. The non-Markovian model is SEIR with log-normal E→I (mean 5.0d, median 4.0d) and I→R (mean 7.5d, median 5.0d) transitions at  $\beta = 0.25$ .

To provide a hardware-transparent measure of the dense  $\mathcal{O}(N + E)$  workload inherent to synchronous tau-leaping, we report throughput in Node-Updates Per Second (NUPS), calculated as  $(N \times \text{steps})/\text{wall-clock time}$ . Because the GPU evaluates all  $N$  nodes every step regardless of how many transitions occur, NUPS reflects the true computational workload. For exact event-driven baselines, we report realized state transitions per second. The Markovian engine achieves  $1.95 \times 10^7$  realized transitions/sec at  $N = 10^6$  — comparable to c-GEMF (Sahneh et al., 2017) at the same



**Table 6**

Throughput decomposition isolating algorithmic and hardware contributions at  $N = 10^6$ ,  $d = 8$ . All numbers are measured on the reference A100 node; the repository README gives the per-row CSV source behind each measurement. The GPU eager entry is converted from 672.9 steps/s to NUPS via  $N \times \text{steps/s}$ . Each throughput point is the mean over replicate runs after 2–3 warm-up runs (the per-benchmark trial/warm-up counts are 10/3 for the Scoglio-style scaling curves, 5/2 for the warp-collab and degree-dispatch benches, and 3/0 for the CPU baseline; the repository README names the harnesses). The coefficient of variation across replicates is below 2% on all rows, and the same replication pattern applies to every throughput table in this paper (Tables 6, 8, 2, 11, 10) unless stated otherwise.

| Configuration               | Throughput        | Incremental                 | Factor       |
|-----------------------------|-------------------|-----------------------------|--------------|
| Exact CPU (Phantom Process) | $\sim 70$ trans/s | —                           | Serial exact |
| Approx. CPU (tau-leaping)   | 37.3 Mega-NUPS    | $\sim 5 \times 10^5 \times$ | Algorithmic  |
| GPU unfused eager           | 0.67 Giga-NUPS    | 18 $\times$                 | Parallelism  |
| GPU CG $b=50$ (unfused)     | 1.87 Giga-NUPS    | 2.8 $\times$                | CUDA Graph   |
| GPU Fused CG $b=50$         | 8.09 Giga-NUPS    | 4.3 $\times$                | Fusion       |

scale — but the experimental evaluation that follows focuses on the renewal path since that is the novel contribution of this work, and we treat the Markovian engine as the Inertial/Control-Mode reference of Sections 4 and 4. The asymptotic complexity comparison of all engines is in Table 9.

A critical methodological concern when comparing approximate GPU tau-leaping to exact CPU Gillespie is that the speedup conflates two independent factors: the algorithmic relaxation (exact  $\rightarrow$  approximate) and the hardware acceleration (CPU  $\rightarrow$  GPU). To isolate these contributions, we implemented a CPU tau-leaping baseline using the same RENEWALENGINE class with PyTorch vectorized operations on an 8-core CPU.

*Composition of the CPU baseline.* The “CPU tau-leaping (8-core)” baseline in Figure 6 and Table 6 is *not* a generic or off-the-shelf parallel tau-leaper; it is our own RENEWALENGINE running on the CPU with eight PyTorch threads enabled, which means the CPU curve already benefits from every algorithmic development in this paper that is hardware-agnostic: (i) the same adaptive Bernoulli tau-leaping step of Section 5 with the same  $\epsilon = 0.03$ ,  $\tau_{\max} = 0.1$ ; (ii) the same numerically stable erfcx-based log-normal hazard of Section 5; (iii) the same  $\mathcal{O}(N + E)$  per-step CSR traversal (gather-based, one contiguous neighbor slice per node) as Section 5.4; (iv) the same renewal age reset on transition; (v) the same counter-based PRNG and seed schedule. The three GPU-specific contributions of this paper have no CPU analogue and are therefore absent from the CPU curve: the Triton FLASHNEIGHBOR kernel (the CPU path uses a scatter-based PyTorch implementation that computes the same result), the fused Triton step kernel (no single-launch abstraction exists on CPU), and CUDA-Graph step batching. The degree-aware CSR dispatch (warp-per-node, merge-based) is likewise GPU-only. CPU parallelism is PyTorch’s intra-op threading across 8 cores, which the scatter and elementwise ops actually use. The Fused-CG vs. CPU gap reported in Table 6 is therefore a strict hardware-plus-implementation gap at *matched* algorithm, not an algorithmic-plus-hardware conflation: the CPU column is the strongest tau-leaping baseline we know how to write with PyTorch.

Table 6 decomposes the throughput hierarchy. The exact CPU Phantom Process achieves  $\sim 70$  transitions/sec at  $N = 10^6$ . CPU tau-leaping on the same network achieves  $3.73 \times 10^7$  NUPS (measured on 8 cores with the same  $\epsilon = 0.03$ ,  $\tau_{\max} = 0.1$  configuration as the GPU engine), representing the pure algorithmic gain from replacing sequential exact simulation with synchronous Bernoulli updates. The GPU fused CUDA Graph engine then reaches  $8.09 \times 10^9$  NUPS (8.09 Giga-NUPS, 1-thread-per-node kernel on the ER  $d=8$  graph at  $N = 10^6$ ), a strict 217 $\times$  hardware speedup over the optimized CPU tau-leaping baseline at the same  $N$ , derived entirely from IO-aware kernel fusion and GPU memory-bandwidth optimization.

Figure 6 shows throughput scaling across network sizes. The fused CUDA Graph engine peaks at 5.86 M events/s at  $N = 10^6$  (equivalently 8.09 Giga-NUPS; events/s and NUPS differ by the per-step realized-transitions-per-node factor, which is tied to  $\epsilon$  and the instantaneous infected fraction).

At  $N = 10^7$ , all engines experience a throughput drop as CSR data ( $\sim 640$  MB) exceeds the A100’s 40 MB L2 cache, a phenomenon we term the “L2 cache cliff.” In events/s the fused CUDA Graph engine drops 4.4 $\times$  ( $5.86 \rightarrow 1.32$  M events/s), the unfused CUDA Graph engine drops 2.1 $\times$  ( $1.81 \rightarrow 0.85$  M events/s), and the unfused eager engine drops 1.75 $\times$  ( $1.32 \rightarrow 0.76$  M events/s); the fused variant is hit hardest at the cliff because it was most bandwidth-saturated beforehand. Replacing binary state checks with continuous age-dependent infectivity (Eq. 8) is nearly free once the



**Table 7**Convergence sweep: tau-leaping accuracy vs. exact Gillespie ( $N = 1000$ , 100 runs, mean  $\pm$  95% CI).

| $\epsilon$ | Peak $I$          | Final $R$         | Steps | Time (s) |
|------------|-------------------|-------------------|-------|----------|
| 0.005      | $0.379 \pm 0.008$ | $0.935 \pm 0.019$ | 5912  | 1.94     |
| 0.01       | $0.383 \pm 0.008$ | $0.934 \pm 0.019$ | 2912  | 0.96     |
| 0.03       | $0.378 \pm 0.011$ | $0.925 \pm 0.026$ | 995   | 0.33     |
| 0.05       | $0.380 \pm 0.011$ | $0.925 \pm 0.026$ | 696   | 0.23     |
| 0.1        | $0.379 \pm 0.011$ | $0.923 \pm 0.026$ | 523   | 0.17     |
| Exact      | 0.314             | 0.863             | —     | —        |

infectivity write is fused into the main kernel: +1.8% wall-clock time on a coalesced regular graph and +0.1% on a scale-free graph (Table 10, discussed in Section 7). A naive pre-pass implementation is 25–50 $\times$  more expensive and makes the abstraction non-free on regular graphs.

To evaluate the framework under extreme degree heterogeneity, we benchmarked the fused CG engine on a Barabási–Albert scale-free network ( $N = 10^6$ ,  $m = 4$ , yielding average degree  $d \approx 8$  for an apples-to-apples memory footprint comparison with the ER baseline). Under the default 1-thread-per-node kernel, throughput dropped from 7.6 to 0.44 Giga-NUPS: hub nodes (maximum degree 3,870,  $D_{\max}/D_{\text{avg}} \approx 484$ ) induce severe warp divergence during CSR traversal, destroying memory coalescing. To close this gap we implemented two alternative CSR traversal strategies (Section 5.5, Table 2); the best of these (edge-partitioned merge-based load balancing) reaches 2.0 Giga-NUPS on the same BA workload, a 4.5 $\times$  speedup over the 1-thread-per-node kernel on scale-free graphs while preserving the peak throughput on regular graphs via runtime dispatch.

We validate the fused kernel against an exact non-Markovian Gillespie simulator (Boguñá et al., 2014) on a standard SEIR benchmark ( $N = 1000$ , Erdős–Rényi graph with  $d = 8$ ,  $\beta = 0.25$ , log-normal transitions, 100 independent runs). Figure 7 shows close agreement for all four SEIR compartments.

Table 7 presents a convergence sweep of the tolerance parameter  $\epsilon \in [0.005, 0.1]$ . The discrepancy in peak infection ( $\sim 6\%$ ) and final attack rate ( $\sim 7\%$ ) relative to the exact baseline does not detectably decrease as  $\epsilon \rightarrow 0$  across the sweep: Figure 11 shows that the per-run bootstrap 95% CIs overlap across adjacent  $\epsilon$  values, and the Pareto CIs of Figure 13 confirm the same pattern on both axes. We interpret this as a structural bias inherent to synchronous Bernoulli updates on contact networks: because all transitions within  $\Delta t$  are applied simultaneously rather than sequentially, the temporal correlation of the expanding epidemic wavefront is slightly altered. Given that empirical epidemiological parameters routinely exhibit uncertainties exceeding 20%, this bounded bias is a favourable tradeoff. We use  $\epsilon = 0.03$  as the default operating point throughout the main text because it sits in the flat portion of the Pareto curve while remaining conservative for publication-quality single runs; Appendix C discusses the  $\epsilon = 0.1$  regime for bulk ensemble sampling and the boundary conditions under which  $\epsilon < 0.03$  is worth the compute.

Figure 8 and Table 8 characterize the computational profile on the A100 (19.5 TFLOPS FP32, 2039 GB/s HBM bandwidth; ridge point at 9.56 FLOPs/byte). All configurations operate in the memory-bound regime (left of the ridge). The fused kernel achieves 555 GFLOPS at arithmetic intensity 0.42, which is  $555/(0.42 \times 2039) = 65\%$  of the memory-bound ceiling evaluated at the fused kernel’s own AI, equivalent to sustaining  $\approx 1320$  GB/s out of the A100’s 2039 GB/s HBM peak — a strong bandwidth-utilisation number for an irregular CSR workload with divergent hazard evaluation. Said differently: if we reported efficiency against the unfused baseline’s AI of 0.66 (as one might do to answer “how much of the pre-fusion roofline did the fused kernel recover?”), the same 555 GFLOPS would read as 41% of  $0.66 \times 2039 = 1346$  GFLOPS; that is the number the earlier draft quoted and it is not wrong, but it measures a comparison against the wrong ceiling. Table 8 now reports both. The arithmetic intensity decreases from 0.66 to 0.42 because block-scalar sparsity removes wasted erfcx FLOPs for inert blocks faster than fusion removes memory traffic; at the fused AI essentially all compute is useful. Layering mixed-precision storage on top of the fused kernel (Section 5.7) shifts the operating point up-and-right along the memory-bound line: the AI climbs from 0.42 to  $\approx 0.62$  because the same algorithmic FLOPs are now amortised over  $\sim 32\%$  fewer bytes of per-step HBM traffic, and the achieved throughput climbs by the measured 1.15 $\times$  lift from 555 to 638 GFLOPS. This is the expected signature of a memory-bound kernel being pushed toward (but not past) the ridge by a byte-band compression; the gain is sub-linear in the byte reduction because kernel launch, index arithmetic, and rate-buffer writes do not scale with storage dtype.

**Table 8**

Roofline benchmark results on A100. CG = CUDA Graph,  $b$  = batch size. “Eff. at own AI” is GFLOPS/(AI  $\times$  2039), the fraction of the memory-bound ceiling evaluated at the configuration’s actual arithmetic intensity. “Eff. vs unfused AI” is GFLOPS/(0.66  $\times$  2039) = GFLOPS/1346 GFLOPS, the fraction of the pre-fusion memory-bound ceiling — useful for comparing fused and unfused kernels against a single reference, but unfavourable to the fused kernel because its effective AI is lower after block-scalar sparsity. Peak bandwidth utilisation is GFLOPS/AI (last column). <sup>†</sup> The mixed-precision Fused CG row is derived from the measured 1.15 $\times$  throughput lift on the same ER  $d=8$ ,  $N = 10^6$  run (Section 5.7, Table 5) and from the  $\sim 32\%$  per-step memory-traffic reduction that follows directly from the state/age/infectivity/weights dtype changes (FLOPs count is unchanged since all kernel math remains fp32). The narrower working set pushes the operating point up and right on the roofline (Figure 8); bandwidth utilisation decreases because the kernel now has less memory work to issue per step rather than because it becomes less efficient.

| Configuration                 | AI   | GFLOPS | ms/step | Eff. at own AI | Eff. vs unfused AI | BW used          |
|-------------------------------|------|--------|---------|----------------|--------------------|------------------|
| Baseline (unfused)            | 0.66 | 49.5   | 1.49    | 3.7%           | 3.7%               | 75 GB/s          |
| CG $b=50$ (unfused)           | 0.66 | 137.0  | 0.54    | 10.2%          | 10.2%              | 208 GB/s         |
| Fused CG $b=50$               | 0.42 | 555    | 0.12    | <b>64.8%</b>   | 41.2%              | <b>1321 GB/s</b> |
| Fused CG + mixed <sup>†</sup> | 0.62 | 638    | 0.10    | 50.5%          | 47.4%              | 1029 GB/s        |

*Validation scope.* We validate the fused renewal kernel on SEIR because SEIR exercises all three transition types used by the framework: edge-mediated Markovian  $S \rightarrow E$ , non-Markovian renewal  $E \rightarrow I$ , and non-Markovian renewal  $I \rightarrow R$ . Section 6.1 adds a complementary Markovian validation on the canonical SIS and SIR benchmarks. Extending the renewal kernel to non-Markovian SIS/SIR requires only edits to the compartment transition map; the CSR traversal, hazard kernel, Bernoulli sampler, CUDA Graph capture path, and auto-dispatch logic are all model-agnostic.

### 6.1. Markovian engine validation (SIS / SIR)

Although the renewal engine is the novel contribution of this paper, the dual-engine framing also requires that the companion Markovian engine (Section 4) is independently validated — both because Markovian SIS/SIR are the canonical benchmarks for the control-theoretic uses of this framework (Nowzari et al., 2016; Watkins et al., 2019; Preciado et al., 2014), and because the Markovian engine is the reference implementation behind the CPU tau-leaping baseline discussed earlier in this section.

We run the same single-graph validation protocol used for the renewal engine, but against an exact *Doob–Gillespie direct-method* reference (not the generalised non-Markovian Gillespie, which degenerates to Doob–Gillespie on exponential holding times). Both the FlashSpread Markovian engine and the exact simulator are run on an Erdős–Rényi graph with  $N = 10^3$ ,  $d = 8$ ,  $\beta = 0.25$ , an initial seed of 10 infected nodes, recovery rate  $\delta = \gamma = 0.15$ , and 100 independent Monte Carlo runs; the tau-leaping sampler uses `max_prob = 0.1` and `theta = 0.01`, which are the defaults provided by the public engine constructor. Figure 9 overlays the two simulators on SIS (endemic steady state, left panel) and SIR (single epidemic wave, right panel). On both models the tau-leaping ensemble mean lies inside the exact Gillespie 25–75% inter-quartile band at every time point of the 100-point sample grid without any further tuning of the sampler.

Appendix C.7 gives the full protocol, reproducibility pointer, and the deeper discussion of the regime separation: SIS tests long-horizon balance between infection and recovery events, SIR tests a single-wave takeoff with an explicit end-state. Combined with the non-Markovian SEIR validation earlier in this section and the multi-topology renewal sweep of Appendix C.4, the framework is validated on all three of SIS, SIR, and SEIR against an exact reference within the relevant regime (Doob–Gillespie for the two Markovian models, generalised non-Markovian Gillespie for SEIR).

### 6.2. Reproducibility

Every result in this paper is reproducible from the repository at <https://github.com/Shakeri-Lab/FlashSpread> against the commit tagged `jocs-submission`. The reference environment, the per-figure and per-table script map with expected wall-clock budgets on the reference A100 node, and the per-curve CSV sources behind Figure 6 and Table 6 are all maintained in the repository README (“Reproducing the Paper” section) so that they stay in sync with the code rather than the manuscript. Random seeds are fixed across scripts, so the JSON and figure artefacts regenerate bitwise-identically on the reference node.

**Table 9**

Complexity comparison of simulation strategies.

| Algorithm         | Per-Step                                | Parallel      |
|-------------------|-----------------------------------------|---------------|
| c-GEMF / FastGEMF | $\mathcal{O}(D_{\max} \log N)$          | Serial        |
| Phantom Process   | $\mathcal{O}(N + D_{\max})$             | Serial        |
| Markov (Inertial) | $\mathcal{O}(K \cdot D_{\text{avg}}/P)$ | $P \sim 10^4$ |
| Markov (Control)  | $\mathcal{O}((N + E)/P)$                | $P \sim 10^4$ |
| Renewal (Fused)   | $\mathcal{O}((N + E)/P)$                | $P \sim 10^4$ |

**Table 10**

Throughput cost of the source-node age-dependent shedding approximation (Fused CG,  $b = 50$ ,  $N = 10^6$ , mean degree 8). “Pre-pass” runs `erfcx` over all  $N$  nodes in PyTorch before the fused kernel; “in-kernel” fuses the infectivity write into the fused kernel’s tail so only lanes that will be  $I$  next step pay the `erfcx` cost.

| Graph                       | Implementation | Constant $\beta$ | Age-dep. $s(\tau)$ | Slowdown |
|-----------------------------|----------------|------------------|--------------------|----------|
| Regular ( $d = 8$ )         | Pre-pass       | 7.10 G-NUPS      | 4.88 G-NUPS        | +45.5%   |
| Regular ( $d = 8$ )         | In-kernel      | 7.61 G-NUPS      | 7.48 G-NUPS        | +1.8%    |
| Barabási–Albert ( $m = 4$ ) | Pre-pass       | 0.434 G-NUPS     | 0.416 G-NUPS       | +4.2%    |
| Barabási–Albert ( $m = 4$ ) | In-kernel      | 0.435 G-NUPS     | 0.434 G-NUPS       | +0.1%    |

## 7. Discussion

The dual-engine architecture reflects a fundamental hardware duality (Table 9). The Markovian engine exploits event-driven sparsity at  $\mathcal{O}(K \cdot D_{\text{avg}})$ ; the renewal engine faces inherently dense, memory-bandwidth-limited workloads requiring IO-aware kernel fusion. The appropriate strategy for each regime is fundamentally different, validating the dual-engine design.

A key conceptual contribution is recognizing that our tau-leaping  $\Delta t_{\max}$  serves the same role as the Phantom-Process’s phantom rate  $\lambda_0$  (Vajdi, 2020): both mechanisms force periodic hazard re-evaluation when rates change continuously. The difference is computational paradigm: Phantom Process is exact and serial; tau-leaping is approximate and massively parallel.

The layered optimization decomposition (Table 6) reveals a striking latency-to-bandwidth crossover. At  $N \leq 10^4$ , CUDA Graph batching provides 5 $\times$  while fusion provides only 3.5 $\times$ . At  $N \geq 10^5$ , the relationship reverses: reducing VRAM traffic via kernel fusion becomes more important than reducing dispatch overhead. When both are combined, execution reaches the irreducible CSR traversal cost of  $8E$  bytes per step.

The source-node approximation (Section 5.3) effectively acts as a zero-cost abstraction, but only because its overhead is itself fused into the main kernel. A naive implementation that precedes the fused kernel with a dense PyTorch infectivity pre-pass (one `torch.special.erfcx` call per node, masked to  $I$  nodes afterward) increases overall wall-clock time by 45.5% on a regular degree-8 graph with highly coalesced memory access: the dense  $\mathcal{O}(N)$  sweep through `erfcx` runs at a point where the SM ALUs are *not* idle, and the “shadow of memory latency” argument does not apply. Moving the infectivity update into the fused kernel itself — each thread writes  $\beta \cdot h_{IR}(\tau'_i)$  (or  $\beta$  for constant shedding) to the next step’s `infectivity` buffer immediately after computing its new age  $\tau'_i$ , guarded by block-scalar sparsity so that blocks with no  $I$ -successor skip the `erfcx` entirely — reduces the overhead to +1.8% on the regular graph and +0.1% on a Barabási–Albert scale-free graph of the same size and mean degree (Table 10). The remaining arithmetic genuinely does execute in the shadow of CSR memory traffic. This is also why the pre-optimization and post-optimization constant- $\beta$  throughputs differ slightly (7.1 vs. 7.6 Giga-NUPS on the regular graph): eliminating the per-step pre-pass removes one kernel launch and the associated read-modify-write on the infectivity buffer.

Several directions remain open. GPU memory limits network size to  $N \lesssim 10^8$ ; multi-GPU domain decomposition would be needed beyond this scale. The current implementation assumes fixed network topology; temporal networks with independently forming and breaking edges would require per-edge age tracking at  $\mathcal{O}(E)$  cost, negating the source-node approximation. The auto-dispatch strategy of Section 5.5 addresses the classic warp-divergence problem on

scale-free graphs (4.5 $\times$  over the default kernel on BA at  $N = 10^6$ ) via edge-partitioned merge-based load balancing (Merrill and Garland, 2016); a further level would be an intra-warp segmented reduction (using low-level CUDA `__shfl_up_sync` primitives (NVIDIA Corporation, 2024)) that collapses same-source partial pressures inside each warp so that only the leader and tail lanes of a segment issue an `atomicAdd` to the shared pressure buffer, eliminating most hub contention. We omitted this optimisation from the public release because it introduces a custom CUDA C++ dependency and a `torch.utils.cpp_extension` build system, trading the current pure-Triton / pip-installable distribution for a back-of-envelope  $\approx 1.3$ – $1.8\times$  speedup in one specific topology regime (BA-hub). The range is an upper-bound estimate, not a measurement: it follows from Amdahl’s-style accounting on our observed atomic-serialisation fraction on BA hubs (the merge kernel’s `atomic_add` into the shared pressure buffer currently sustains 1–2 GB/s on same-destination contention versus the  $\sim 1.3$  TB/s headroom used elsewhere in the step) combined with the expected fraction of same-source runs per warp produced by the row-sorted CSR chunk. A prototype implementation would be required to confirm it; we have not produced one. The tradeoff is unfavourable for a user-facing framework whose portability (including future AMD ROCm targets, which Triton handles natively) is a first-order design property, so we flag the segmented-reduction path as *known-useful future work* rather than an omitted optimisation. Finally, Appendix C characterises the  $\epsilon$ -independent structural-bias floor empirically; a formal convergence analysis as a function of  $\epsilon$ , network spectral gap, and degree distribution remains open and would tighten the honesty of the current “does not detectably decrease” framing into a provable bound.

## 8. Conclusion

We presented FLASHSPREAD, a GPU framework for non-Markovian epidemic simulation on networks. The fused renewal engine consolidates the entire per-step pipeline into a single Triton kernel, reaching 8.09 Giga-NUPS on a  $10^6$ -node regular graph on an A100 (217 $\times$  over the optimised CPU tau-leaping baseline at the same  $N$ ). Degree-aware CSR dispatch — auto-selected from  $D_{\max}/D_{\text{avg}}$  at engine construction — adds a 4.5 $\times$  recovery on scale-free graphs without regressing the regular-graph peak (Section 5.5, Appendix B). The source-node approximation (Section 5.3) makes age-dependent shedding essentially free once the infectivity update is folded into the fused kernel:  $\leq 2\%$  overhead on both regular and scale-free graphs (Table 10).

Validation against an exact non-Markovian Gillespie reference shows a  $\sim 6\%$  structural-bias floor in peak- $I$  that does not detectably decrease across two decades of tolerance on our benchmarks (Appendix C). Combined with the CPU tau-leaping baseline that cleanly isolates the algorithmic relaxation from the hardware speedup, this supports the framework as a rigorous foundation for ensemble forecasting and policy evaluation under biologically realistic non-Markovian dynamics at scales that were previously confined to the CPU era.

## Declaration of competing interest

The authors declare no competing interests.

## Declaration of generative-AI use

During the preparation of this manuscript the authors used large-language-model assistants (Anthropic Claude and OpenAI ChatGPT) for prose revision, figure-drafting support, and code review of the accompanying software. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

## Acknowledgments

Computational resources were provided by the University of Virginia Research Computing.

## CRedit authorship contribution statement

**Heman Shakeri:** Conceptualization, Methodology, Software, Writing – Original Draft, Visualization, Funding Acquisition, Supervision, Project Administration. **Behnaz Moradi-Jamei:** Methodology, Writing – Review & Editing. **Aram Vajdi:** Methodology, Validation, Writing – Review & Editing. **Ehsan Ardjmand:** Methodology, Writing – Review & Editing.

## A. Triton erfcx approximation

Because the GPU kernel compiler (Triton) lacks a native `erfcx` instruction, we implemented a piecewise approximation inside the fused kernel that combines a direct identity for  $|z| \leq 3.5$  with a four-term asymptotic expansion for  $|z| > 3.5$ . For  $z \geq 0$ :

$$\text{erfcx}(z) \approx \begin{cases} e^{z^2}(1 - \text{erf}(z)) & |z| \leq 3.5 \\ \frac{1}{z\sqrt{\pi}} \left( 1 - \frac{1}{2z^2} + \frac{3}{4z^4} - \frac{15}{8z^6} \right) & |z| > 3.5 \end{cases} \quad (9)$$

For  $z < 0$ , we use the identity  $\text{erfcx}(z) = 2e^{z^2} - \text{erfcx}(-z)$ . For  $|z| > 9$ , the asymptotic form is applied unconditionally to prevent float32 overflow in the  $e^{z^2}$  computation ( $e^{81} \approx 5.3 \times 10^{35}$  is safe;  $e^{88.7}$  exceeds float32 range). As  $\tau \rightarrow 0$  after a renewal reset,  $z \rightarrow -\infty$  and  $\text{erfcx}(z) \rightarrow \infty$ , yielding  $h_{LN} \rightarrow 0$ , which correctly reflects the biological reality that transition probability is zero immediately after entering a state.

We validate the approximation on a dense fp32-safe grid  $z \in [-9, 30]$  (10,001 points); the upper negative bound is set by  $\exp(z^2)$  overflowing single-precision at  $|z| \gtrsim 9.4$ , not by a limitation of the formula. The measured maximum relative error is  $\sim 4 \times 10^{-2}$  at  $z \approx 3.5$ , driven by catastrophic cancellation in the fp32 evaluation of  $1 - \text{erf}(z)$  precisely at the branch-switch, and drops to  $\sim 6 \times 10^{-3}$  away from the branch boundary. Because these errors enter the Bernoulli step as  $1 - \exp(-\lambda_i \Delta t)$  with  $\lambda_i \Delta t \lesssim \epsilon = 0.03$ , the induced bias in transition probability stays well below the structural-bias floor of synchronous tau-leaping itself ( $\sim 6\%$  peak- $I$  and  $\sim 7\%$  final- $R$ ), so the approximation is comfortably within the tolerance of the stochastic integrator. A unit test (cited in the repository README) reproduces these bounds and guards against regressions. Random number generation uses Triton’s native PRNG, seeded via a 64-bit host-managed step counter that increments by 1 per simulation step, preventing pattern repetition for over  $2^{31}$  steps.

## B. Degree-heterogeneous graphs and auto-dispatch

The 1-thread-per-node fused kernel introduced in Section 5.4 is optimal when the degree distribution is narrow, but degrades sharply on heavy-tailed topologies (scale-free, preferential-attachment, many real social networks). This appendix documents the two alternative CSR traversal kernels we provide, the auto-dispatch heuristic, and the measured gains, all consistent with Section 5.5 and Table 2.

### B.1. Why the default kernel stalls on hubs

In the 1-thread-per-node kernel, thread  $i$  in a block of 128 threads iterates its own  $D_i$  neighbors with a scalar `while` loop; all 128 threads in the block proceed to the per-node tail only when every thread’s `while` has terminated. The block’s wall-clock runtime is therefore  $\mathcal{O}(\max_{i \in \text{block}} D_i)$ . On a Barabási–Albert graph with  $N = 10^6$  and  $m = 4$  the largest measured degree is  $D_{\max} = 3,870$  ( $D_{\max}/D_{\text{avg}} = 484$ ), so any block containing even one hub stalls for 3,870 CSR iterations while 127 other threads have long since finished their degree-4 lists. Memory coalescing is also destroyed: the 128 threads are reading very different `col_ind` offsets after the first few iterations. Measured fused-CG throughput collapses from 7.6 Giga-NUPS on a regular degree-8 graph to 0.45 Giga-NUPS on BA at the same  $N$ .

### B.2. Warp-per-node kernel

The first remediation cooperates within a warp: thread layout becomes `[NODES_PER_BLOCK, LANES_PER_NODE]` with `LANES_PER_NODE = 32`, one warp per node. Each warp iterates the node’s neighbor list in chunks of 32 edges, using a 2D register tile for partial pressures; at the end a `tl.sum` along the lane axis collapses the tile to a per-node scalar. Hub traversal now takes  $\lceil D_{\max}/32 \rceil$  warp-parallel iterations rather than  $D_{\max}$  serial ones.

The per-node tail (hazard evaluation, Bernoulli sampling, transition, next-infectivity write) re-uses the 1D code path of the baseline kernel on a `NODES_PER_BLOCK`-element tensor; only those few lanes produce meaningful writes, so a moderate amount of block-wide ALU/memory bandwidth is wasted there. The tradeoff is favorable for hub-containing blocks but loses on uniform-degree graphs because the CSR coalescing that the baseline enjoys is given up (most lanes do useless work in any given chunk when  $D_i \ll 32$ ).

We verified bit-identical per-step parity of the warp kernel against the baseline on both a regular degree-8 graph and a BA graph at  $N = 10^4$  over 50 steps, using the deterministic `tl.rand(seed + step_id, node_id)`



RNG pattern shared across kernels. Zero mismatches were observed in `state`, `age`, `infectivity`, or `rates` at any step.

### B.3. Edge-partitioned merge-based kernel

The second remediation is a merge-path load-balancer along the lines of Merrill and Garland (2016): each program processes a fixed chunk of `EDGES_PER_BLOCK` contiguous edges (not nodes), independent of which nodes those edges belong to. Each thread in the program handles one edge and performs:

1. Load `(col_ind[e], weights[e], infectivity[col_ind[e]])`.
2. Recover the source node id  $n$  such that `row_ptr[n] ≤ e < row_ptr[n + 1]` via a fully unrolled binary search of depth  $\lceil \log_2(N + 2) \rceil$  over `row_ptr`.
3. Atomic-add the weighted contribution `infectivity[col_ind[e]] · weights[e]` to a pressure scratch buffer at index  $n$ .

A second lightweight kernel (`_flash_renewal_tail_kernel`) then reads the pressure buffer and runs the unchanged per-node tail: hazard, Bernoulli, transition, next-infectivity write. Because the two kernels share the same RNG pattern as the single-kernel variants, they produce statistically identical trajectories.

Load balance across blocks is now perfect regardless of degree skew — every block does exactly `EDGES_PER_BLOCK` worth of work. The price is (i) one atomic add per edge (contention dominated by hubs, but still much faster than the baseline’s serial hub traversal), (ii) a scratch buffer read/write of  $4N$  bytes, and (iii) one extra kernel launch per step. The scheme pays off when degree skew is large enough that the baseline’s idle-warp waste exceeds these fixed costs.

### B.4. Auto-dispatch

At engine construction FLASHSPREAD inspects `row_ptr` once, computes  $\rho = D_{\max}/D_{\text{avg}}$ , and selects:

$$\text{strategy}(\rho) = \begin{cases} \text{thread} & \rho < 4, \\ \text{warp} & 4 \leq \rho < 50, \\ \text{merge} & \rho \geq 50. \end{cases} \quad (10)$$

The thresholds are calibrated from the sweep in Table 11. The user can override via an explicit `csr_strategy` argument. The dispatch cost is one pass over `row_ptr` at construction ( $\mathcal{O}(N)$ , amortized trivially over every subsequent step).

### B.5. Parity and throughput

**Parity.** We ran the three strategies on  $N = 10^4$  regular-degree-8 and BA- $m=4$  graphs for 50 tau-leaping steps with a deterministic seed. The `thread` and `warp` kernels produced bit-identical per-node outputs (`state`, `age`, `infectivity`, `rates`) at every step. The `merge` kernel uses non-deterministic atomic-add ordering and therefore can differ by fp rounding at the LSBs of the pressure buffer; over our 50-step window on both graphs the SEIR population counts matched bit-exactly (zero delta in all four bins) because no Bernoulli draw landed within fp ULP of its threshold.

**Throughput.** Table 11 shows the full tunable sweep on both graphs at  $N = 10^6$  (Fused CG  $b = 50$ , A100, same experimental setup as Section 6).

Key observations:

- On the regular graph ( $\rho = 2$ ), every alternative strategy is strictly worse than `thread`; auto-dispatch correctly chooses `thread`, so the user pays nothing for having the machinery available.
- On BA ( $\rho = 484$ ), `merge` outperforms `warp` by  $\sim 50\%$  at the best tuning (EPB = 1,024 wins over NPB = 8), and both outperform `thread`.
- The `warp` strategy is not currently a winner in the auto-dispatch decision (`merge` strictly dominates it for  $\rho \geq 50$  in our sweep), but we retain it for two reasons: its per-block register-tile accumulation is atomic-free and deterministic bit-to-bit (useful for reproducibility-critical users), and its optimal regime  $4 \leq \rho < 50$  covers medium-heterogeneity graphs on which the `merge` scheme’s atomic contention would be unnecessary overhead.

**Table 11**

Full CSR-strategy throughput sweep on ER and BA at  $N = 10^6$ , Fused CG,  $b = 50$ , A100. NPB = NODES\_PER\_BLOCK (warp); EPB = EDGES\_PER\_BLOCK (merge). Bold entries denote the strategy auto-dispatch picks for that graph.

| Strategy                  | Tunable     | Regular $d = 8$<br>G-NUPS | BA $m = 4$<br>G-NUPS |
|---------------------------|-------------|---------------------------|----------------------|
| thread                    | —           | <b>7.88</b>               | 0.45                 |
| warp                      | NPB = 2     | 0.86                      | 0.75                 |
| warp                      | NPB = 4     | 1.37                      | 1.16                 |
| warp                      | NPB = 8     | 1.70                      | 1.30                 |
| merge                     | EPB = 1,024 | 3.92                      | <b>2.00</b>          |
| merge                     | EPB = 2,048 | 3.44                      | 1.78                 |
| merge                     | EPB = 4,096 | 3.35                      | 1.86                 |
| merge                     | EPB = 8,192 | 1.41                      | 0.84                 |
| <b>Auto-dispatch gain</b> | vs. thread  | 1.00×                     | <b>4.48×</b>         |

- EPB = 1,024 is the sweet spot on both graphs. Smaller EPB increases launch overhead; EPB = 8,192 concentrates too many atomic targets per block and the scheduler can't hide the contention behind other work.

Combining the contributions in Section 5 and Section 5.5, the end-to-end speedup chain on BA at  $N = 10^6$  is: exact CPU Phantom Process (Vajdi, 2020)  $\approx 70$  transitions/sec; CPU tau-leaping  $\approx 37$  M-NUPS ( $5 \times 10^5 \times$  from algorithmic relaxation); 1-thread-per-node fused CG on BA = 0.45 G-NUPS (12× hardware from the naive GPU); auto-dispatched merge-based fused CG on BA = 2.0 G-NUPS (4.5× from degree-aware dispatch). Every factor is strict-hardware (no algorithmic relaxation past the tau-leaping baseline), and the entire chain reproduces end-to-end from the repository; the README names the harness that regenerates these numbers.

## C. Tau-leaping fidelity and compute budgeting

The fused Bernoulli tau-leaping of Algorithm 3 introduces two distinct sources of error relative to the exact continuous-time stochastic process on the network: a *discretization* error from finite  $\Delta t$  that vanishes as  $\epsilon \rightarrow 0$ , and a *structural* error from synchronous per-step updates that does not. This appendix quantifies both on the standard SEIR benchmark ( $N = 10^3$ , ER  $d = 8$ , log-normal  $E \rightarrow I$  and  $I \rightarrow R$ ,  $\beta = 2/d$ , 100 runs), maps the fidelity–compute Pareto frontier, and gives a practical rule for choosing  $\epsilon$  under a given compute budget.

### C.1. Two sources of error

*Discretization.* The per-node Bernoulli step probability  $p_i = 1 - \exp(-\lambda_i \Delta t)$  is exact only when  $\lambda_i$  is constant on  $[t, t + \Delta t)$ . For non-Markovian hazards  $h(\tau)$  the rate varies smoothly within a step; for Markovian rates it changes discretely whenever a neighbor transitions. Both effects contribute an  $\mathcal{O}(\epsilon)$  bias in the expected transition count per step. Reducing  $\epsilon$  tightens the integration at the cost of more steps per trajectory, up to the point where per-step cost becomes negligible and only the structural term remains.

*Structural.* Within a single tau-leap, all nodes sample Bernoulli trials against their *begin-of-step* rate; transitions are then applied simultaneously. This omits the (non-negligible) event that node  $u$  transitions mid-step and alters node  $v$ 's rate before  $v$  samples. Exact Gillespie captures this by ordering events one at a time; tau-leaping does not. The structural bias is therefore a property of synchronous updates on the network and is independent of  $\epsilon$ : it vanishes only in the thermodynamic / mean-field limit, or when the per-node rate is so small that at most one transition per Bernoulli window is a valid approximation.

### C.2. Fidelity metrics

We report four metrics of the ensemble-mean trajectory  $(\hat{S}, \hat{E}, \hat{I}, \hat{R})(t)$  against the exact Gillespie reference  $(S, E, I, R)(t)$ :

- Trajectory  $L_\infty$ :  $\max_{t,s} |\hat{y}_s(t) - y_s(t)|$  over time and compartment.

- Trajectory  $L_2$ :  $\sqrt{(\hat{y}_s(t) - y_s(t))^2}$  over the 501-point sample grid.
- Peak-infection error:  $|\max_t \hat{I}(t) - \max_t I(t)|$ .
- Final-attack-rate error:  $|\hat{R}(T) - R(T)|$  at  $T = 50$ .

For the appendix plots we additionally report a *self-consistency* error against our finest discretization ( $\epsilon = 0.005$ , 100 runs) as a proxy for the discretization component alone, isolated from the structural bias.

### C.3. Empirical convergence

Figure 10 overlays ensemble-mean trajectories of four  $\epsilon$  values. Qualitatively,  $\epsilon = 0.1$  already tracks the  $\epsilon = 0.005$  reference across all four compartments; reductions below  $\epsilon \approx 0.03$  change the curves only imperceptibly at plot resolution.

Figure 11 shows the quantitative picture on log-log axes, with per-run error bars from a 1,000-resample bootstrap over the 100 Monte Carlo trajectories. The peak- $I$  and final- $R$  errors *against exact Gillespie* (solid markers) are statistically indistinguishable across two decades of  $\epsilon$ ; the 95% CI envelopes of adjacent  $\epsilon$  values overlap, so the apparent non-monotonicity in the point estimates is ensemble-sampling noise rather than a true convergence pattern. To convert this visual claim into a quantitative one, we fit  $\log_{10} \text{err} = \alpha \log_{10} \epsilon + c$  across the full  $\epsilon \in [0.005, 0.2]$  sweep and bootstrap the slope  $\alpha$  over 5,000 resamples: the fitted slopes are  $\alpha_{\text{peak-}I} = +0.024$  with 95% CI  $[-0.026, +0.053]$  and  $\alpha_{\text{final-}R} = +0.031$  with 95% CI  $[-0.106, +0.116]$ . Both intervals contain zero, so the hypothesis of no log-linear decrease of the exact-reference error with  $\epsilon$  is not rejected at the 0.05 level; if anything, the point estimates are slightly positive, consistent with the structural bias being an  $\epsilon$ -independent floor rather than a vanishing error term. In contrast, the self-consistency  $L_\infty$  and  $L_2$  metrics (dashed, vs the  $\epsilon = 0.005$  ensemble) decrease monotonically with  $\epsilon$ : this is the pure discretization component. The separation between the two sets of curves is the *structural bias floor*.

### C.4. Robustness across topologies and sizes

The single-graph result above could in principle be an artefact of the particular sparse ER topology or of the  $N = 10^3$  scale. To rule that out, we repeat the sweep over three qualitatively different graph families — sparse Erdős–Rényi ( $d = 8$ ), scale-free Barabási–Albert ( $m = 4$ , average degree  $\approx 8$ ,  $D_{\max}$  growing with  $N$ ), and deterministic fixed-degree ( $d = 8$ , homogeneous) — and three network sizes ( $N \in \{10^3, 10^4, 10^5\}$ ). For each  $(G, N, \epsilon)$  cell we average 20 Monte Carlo trajectories. To keep the ensemble-mean informative, initial seeding is  $\max(10, 0.01N)$  Exposed nodes at  $t = 0$ , well above the stochastic-fadeout threshold on all three topologies; this lets us focus the comparison on the bulk epidemic dynamics rather than the early extinction-prone phase.

Figure 12 shows the resulting  $I(t)/N$  curves. Four properties are robust across the grid:

- At every  $(G, N)$  the  $\epsilon = 0.1$  trajectory lies visually on top of the  $\epsilon = 0.005$  reference; the small gap between them is the discretization component and is insensitive to topology or scale.
- Epidemic *shape* depends on topology. ER and fixed-degree give similar symmetric peaks near  $t \approx 25$ ; BA’s heavy-tailed degree distribution produces an earlier takeoff and a wider peak, because the few high-degree hubs reach saturation faster than the network average.
- Epidemic *size* depends on  $N$ . On ER the peak fraction is near-stationary around 0.37; on BA it drifts slightly with  $N$  because the hub fraction changes. The tau-leaping-to-reference agreement holds at every  $N$  individually.
- The agreement-across- $\epsilon$  story is preserved: the curve at  $\epsilon = 0.1$  tracks the reference to within plot resolution on every panel, which is the main fidelity claim of the appendix, tested against topology and scale simultaneously.

### C.5. Compute-fidelity Pareto

Figure 13 plots mean wall-clock per trajectory against  $\max(\text{err peak-}I, \text{err final-}R)$  versus exact, with bootstrap 95% CIs on both axes. Within those CIs the error metric is statistically flat across  $\epsilon \in [0.005, 0.2]$  while wall-clock scales as  $\sim 1/\epsilon$  below the  $\tau_{\max}$ -clamp region. Any point strictly above the fastest one (top-right of the overlap band) pays compute for no statistically significant fidelity gain, and in particular  $\epsilon = 0.005$  is almost always wasteful on this benchmark.

**Table 12**

Recommended  $\varepsilon$  by compute budget and application. All timings are for  $N = 10^3$  single-trajectory wall-clock on an A100, from the fidelity sweep plotted in Figures 11–13. The err peak- $I$  column is the per-run mean against the exact non-Markovian Gillespie reference produced by the companion MATLAB simulator on the same graph and parameters, with the 95% bootstrap CI in brackets (1,000 resamples over 100 Monte Carlo runs). The CI envelopes overlap between adjacent rows: within finite-ensemble noise the err peak- $I$  is flat across two decades of  $\varepsilon$  while wall-clock varies by a factor of  $\sim 7$ . Reducing  $\varepsilon$  below  $\sim 0.03$  buys compute, not fidelity, on this benchmark, which is why the main text uses  $\varepsilon = 0.03$  as the default;  $\varepsilon = 0.1$  is a viable aggressive-throughput alternative when structural bias is known to dominate and per-trajectory fidelity is not the scoring metric (e.g., bulk RL training). Rows are ordered from fastest to slowest.

| Use case                                       | $\varepsilon$ | err peak- $I$ (95% CI) | wall-clock (95% CI)    |
|------------------------------------------------|---------------|------------------------|------------------------|
| Aggressive throughput (RL training, ensembles) | 0.2           | 6.60% [5.98, 7.42]     | 0.157 s [0.157, 0.158] |
| RL training / bulk ensemble sampling           | 0.1           | 6.59% [5.92, 7.49]     | 0.161 s [0.160, 0.161] |
| Default (forecasts, policy evaluation)         | 0.03          | 6.37% [5.73, 7.33]     | 0.239 s [0.236, 0.241] |
| Publication-quality validation                 | 0.01          | 6.28% [5.82, 6.97]     | 0.558 s [0.547, 0.565] |
| Over-resolved (almost always wasteful)         | 0.005         | 5.88% [5.40, 6.53]     | 1.058 s [1.033, 1.079] |

### C.6. Tuning rules

From the Pareto analysis we extract a simple rule of thumb, summarised in Table 12. The numbers are calibrated to this benchmark; the underlying logic — choose the largest  $\varepsilon$  that keeps you in the flat portion of the curve — transfers to other topologies and parameter regimes.

*When the rule of thumb breaks.* The recommendation  $\varepsilon = 0.1$  is tied to the observation that structural bias dominates here. Two regimes where the discretization term dominates instead, and where  $\varepsilon$  should be reduced towards 0.01:

- *Near-mean-field / large- $N$  regimes*, where the structural bias vanishes (Starnini et al., and the correspondence literature discussed in Section 2). In these regimes discretization is the only error source and  $\varepsilon$  should be chosen directly from the  $\mathcal{O}(\varepsilon)$  Bernoulli integration bound.
- *Highly non-Markovian dynamics with sharply peaked hazards*, where the per-step rate change is large (e.g., log-normal shedding profiles with small  $\sigma$ ). Here the begin-of-step rate is a poor approximation of the integrated rate, and a smaller  $\varepsilon$  is warranted.

A single short run at  $\varepsilon \in \{0.1, 0.03\}$  on any new workload is typically enough to tell which regime one is in: if the two agree to within 1% on the target metric, keep  $\varepsilon = 0.1$ ; otherwise step down to  $\varepsilon = 0.03$  and re-check.

The sweep in this appendix is reproducible from the repository; the README lists the data-generation and plotting scripts.

### C.7. Markovian SIS / SIR validation: exact protocol and reproducibility pointer

Section 6.1 presents the headline result; the IQR-band agreement and regime separation between SIS and SIR are discussed there. This appendix exists only to document the protocol with enough detail to reproduce the run from the public repository, without repeating the result prose in the main text.

**Parameters.** Erdős–Rényi graph with  $N = 10^3$  nodes and average degree  $d = 8$ ,  $\beta = 0.25$ , initial seed of 10 infected nodes, recovery rate 0.15 (either  $\delta$  for SIS or  $\gamma$  for SIR), 100 independent Monte Carlo runs.

**Implementations.** The exact reference is a standard Doob–Gillespie direct-method simulator maintained incrementally; the FlashSpread side uses the public MARKOVIANENGINE with its default adaptive tau-leaping sampler at `max_prob = 0.1` and `theta = 0.01`. No parameter tuning beyond those defaults was performed.

**Reproducibility.** Both simulators, their SLURM wrappers, and the post-processing that produced Figure 9 are listed in the “Reproducing the Paper” section of the repository README against the `jocs-submission` tag.

## D. Data layout, FlashNeighbor, and the GPU memory flow

Section 4 described the FlashNeighbor kernel at a high level: one thread per node, gather-based CSR traversal, no atomics, a single write-back per thread. This appendix fills in the concrete data layout, the per-thread execution

pattern, and the corresponding traffic through the GPU memory hierarchy. These details drive the asymptotic figures quoted in Section 5.4 ( $\sim 64N$  bytes unfused  $\rightarrow \sim 20N$  bytes fused) and the  $\mathcal{O}((N+E)/P)$  per-step complexity claimed throughout the paper.

### D.1. Compressed sparse row (CSR) representation

FLASHSPREAD stores the contact network in Compressed Sparse Row (CSR) form, *indexed by incoming edges*, with a mirrored outgoing CSR maintained only for the Markovian Inertial-Mode sparse update path (Section 4). Figure 14 traces one node through the three arrays.

The layout has three properties that the kernel design exploits:

- *Contiguity.* A node’s neighbors occupy a contiguous slice of `col_ind` and `weights`. A warp of threads processing one node therefore issues a coalesced burst load; a block of threads processing one-node-per-thread produces  $B$  scattered bursts, where  $B$  is the thread-block size.
- *Cacheability.* On an A100, L2 is  $\sim 40$  MB, shared among all SMs. A CSR for  $N = 10^6$  and  $d = 8$  (`col_ind` + `weights`) totals 64 MB and no longer fits; the “L2 cache cliff” at  $N = 10^7$  (Figure 6) is this effect. Auto-dispatch does not change the data layout but changes the *access order*, which is what determines whether the L2 miss pattern stays tractable on scale-free graphs (Appendix B).
- *Memory footprint* is  $4(N+1)$  bytes (`row_ptr`, int32) +  $4E$  (`col_ind`, int32) +  $4E$  (`weights`, fp32) =  $8E + 4(N+1)$  bytes for the incoming copy, doubled when the outgoing copy is present. At  $N = 10^6$ ,  $d = 8$  (directed edge count  $E = 8N = 8 \times 10^6$ ), this is  $8 \cdot 8 \times 10^6 + 4 \cdot (10^6 + 1) \approx 68$  MB per copy (dominated by the  $8E$  edge payload; we abbreviate as “ $\sim 64$  MB” elsewhere in the text when quoting the  $8E$  edge term alone). At  $N = 10^7$  the same formula gives  $\sim 680$  MB per copy, which is where the L2 cache cliff in Figure 6 sets in. The  $N \lesssim 10^8$  single-GPU ceiling quoted in the paper follows directly from doubling this footprint plus  $\mathcal{O}(N)$  per-node state ( $4N + 4N + 4N + 4N$  bytes for `state`, `age`, `infectivity`, `rates`) and fitting in the A100’s 40–80 GB of HBM.

### D.2. FlashNeighbor: per-thread execution inside one fused launch

The kernel maps one GPU thread to one target node  $i$ , with no shared memory and no atomics. Figure 15 shows the data flow for a single warp.

Three implementation details make this efficient:

1. *Write-once output.* The per-thread pressure  $p_i$  is a register-resident fp32 accumulator during the whole loop; it materializes in HBM only once, at kernel end. In the unfused pipeline this single register value became a full  $\mathcal{O}(N)$  intermediate tensor between two kernels.
2. *Indirect gather cost is paid once per edge.* Each edge  $(i, j)$  contributes three memory transactions on the read side: `col_ind[e]` (coalesced), `weights[e]` (coalesced), and `infectivity[col_ind[e]]` (gather, potentially L2-resident if the neighbor  $j$  was recently touched). The gather is the bandwidth-bound operation; the coalesced loads ride in the shadow of its latency.
3. *Block-level sparsity* (Section 5.4) is applied *after* the pressure accumulation, as part of the hazard-evaluation stage in the same launch. The reduction `any_e`, `any_i` is a warp shuffle, essentially free, so the kernel never materialises a mask tensor.

In Appendix B we replace step 1’s “one thread per node” by either “one warp per node” (for  $\rho \geq 4$ ) or “one thread per edge” with binary-search node attribution (for  $\rho \geq 50$ ). The *data layout* is identical in all three cases; only the thread-to-work mapping changes.

### D.3. Flow through the GPU memory hierarchy

The reason kernel fusion matters on this workload is not FLOP count but HBM traffic. Figure 16 contrasts the legacy unfused pipeline (five separate kernels, each materialising an  $\mathcal{O}(N)$  intermediate tensor to HBM) with the fused kernel (all intermediates stay in SM registers for the lifetime of one per-node pipeline).

Three quantitative observations follow:



**Per-step byte counting.** With all intermediates in registers the dominant HBM traffic per step is  $4N$  (state read) +  $4N$  (age read) +  $4N$  amortised for the `infectivity` read (the load is a scatter through `col_ind`, so its HBM traffic is bounded by  $4N$  only when the neighbour working set sits in L2; otherwise L2 misses inflate it up to  $4E$  in the worst case) +  $4N$  (`next_state` write) +  $4N$  (`next_age` write), giving  $\sim 20N$  bytes of node-level traffic. CSR data adds  $8E$  bytes of edge-level traffic, most of which is served from L2 until the L2 cache cliff is reached. The ratio  $20N/(20N + 8E) = 20/(20 + 8d)$  places node-state traffic at about 24% of the total for  $d = 8$ ; beyond that threshold the L2-serviceable CSR stream dominates.

**L2 acts as the second-chance cache for neighbour lookups.** The gather `infectivity[col_ind[e]]` is a scattered load whose working set is bounded by  $N \text{ fp32} = 4N$  bytes. At  $N = 10^6$  this fits in L2 on any current datacenter GPU; at  $N = 10^7$  it begins to spill, explaining the  $4.5\times$  throughput drop of the fused engine (and  $20\times$  drop of unfused variants) between the two scales.

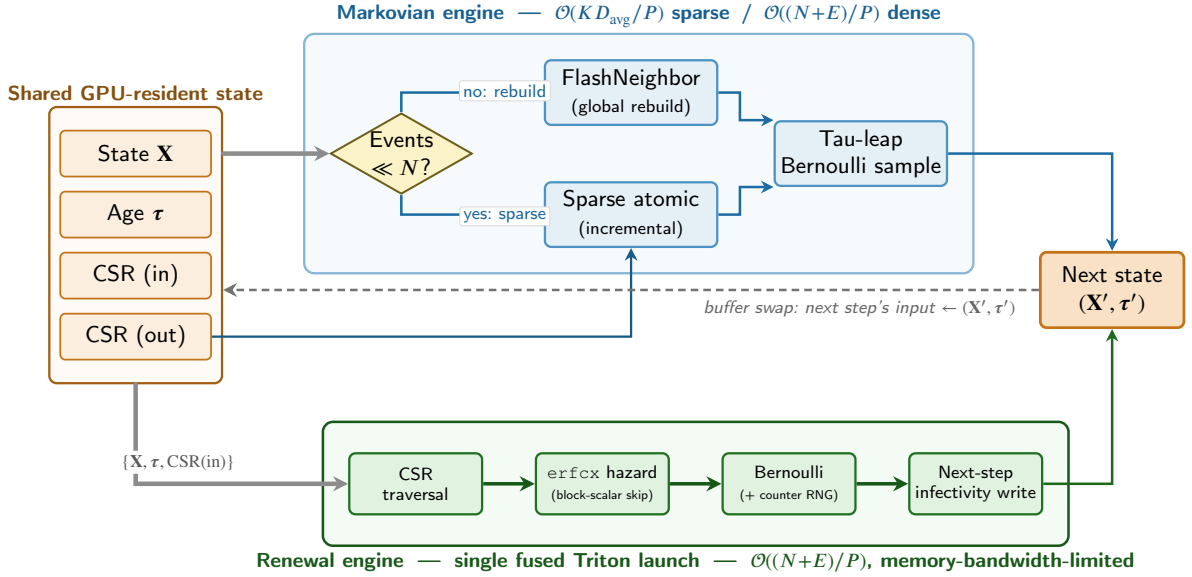
**No shared memory pressure.** Because each target node is owned by exactly one thread and because the pressure accumulator is a scalar float, no shared memory is ever allocated by the fused kernel. This leaves the SM's shared memory available to the block scheduler for occupancy improvements; on an A100, increasing block size from 128 to 256 changes occupancy but not working-set size because no shared-memory spill ever occurs. This is part of why the fused kernel reaches 41% of the reduced memory-bound ceiling (Table 8) rather than stalling on occupancy: the SM is bandwidth-limited, not pressure-limited.

The block-level skip on `erfcx` (Section 5.4) interacts with this traffic accounting as follows. The reduction that computes `any_e` and `any_i` is a block-level reduction (cheap in absolute terms compared to the  $\sim 55$ -FLOP `erfcx` evaluation it guards), and when a block's active mask is all zero the block skips the hazard *computation* but not its *data traffic*: the state and age of those nodes were still read from HBM. The  $\sim 4\times$  “fusion” gain plotted in Figure 8 therefore decomposes into a  $\sim 3\times$  data-traffic reduction (this appendix) and a residual  $\sim 1.3\times$  from the compute-sparsity of block-level skips on inert blocks — the two effects are complementary rather than double-counted.

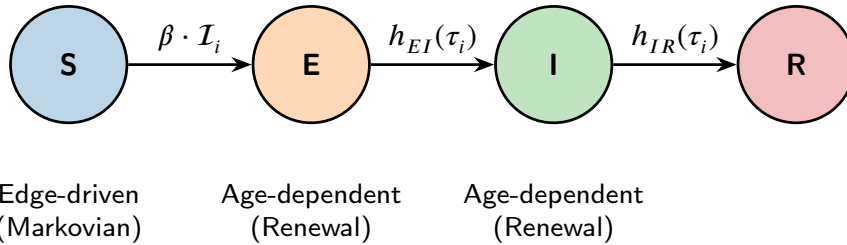
## References

- Anderson, R.M., May, R.M., 1991. Infectious diseases of humans: dynamics and control. Oxford university press.
- Arai, J., Shiokawa, H., Yamamuro, T., Onizuka, M., Iwamura, S., 2016. Rabbit order: Just-in-time parallel reordering for fast graph analysis, in: IEEE International Parallel and Distributed Processing Symposium (IPDPS), IEEE. pp. 22–31.
- Backer, J.A., Klinkenberg, D., Wallinga, J., 2020. Incubation period of 2019 novel coronavirus (2019-ncov) infections among travellers from wuhan, china, 20–28 january 2020. *Eurosurveillance* 25, 2000062.
- Barrat, A., Barthélemy, M., Vespignani, A., 2008. Dynamical processes on complex networks. Cambridge University Press.
- Bisset, K.R., Chen, J., Feng, X., Kumar, V.S.A., Marathe, M.V., 2009. Parallel algorithms for simulating interacting individuals with infections on GPUs, in: International Conference on Parallel Processing, IEEE. pp. 223–230.
- Boguñá, M., Lafuerza, L.F., Toral, R., Serrano, M.Á., 2014. Simulating non-Markovian stochastic processes. *Physical Review E* 90, 042108.
- Cao, Y., Gillespie, D.T., Petzold, L.R., 2006. Efficient step size selection for the tau-leaping simulation method. *The Journal of Chemical Physics* 124, 044109.
- Chatterjee, A., Vlachos, D.G., Katsoulakis, M.A., 2005. Binomial distribution based  $\tau$ -leap accelerated stochastic simulation. *The Journal of Chemical Physics* 122, 024112.
- Cure, S., Pflug, F.G., Pigolotti, S., 2025. Fast and exact stochastic simulations of epidemics on static and temporal networks. *PLOS Computational Biology* 21, e1013490.
- Dao, T., Fu, D., Ermon, S., Rudra, A., Ré, C., 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35, 16344–16359.
- Dobson, S., 2017. Epydemic: Epidemic simulations on networks in python.
- Feng, M., Cai, S.M., Tang, M., Lai, Y.C., 2019. Equivalence and its invalidation between non-Markovian and Markovian spreading dynamics on complex networks. *Nature Communications* 10, 3748.
- Gillespie, D.T., 1976. A general method for numerically simulating the stochastic time evolution of coupled chemical reactions. *Journal of computational physics* 22, 403–434.
- Gillespie, D.T., 1977. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry* 81, 2340–2361.
- Gillespie, D.T., 2001. Approximate accelerated stochastic simulation of chemically reacting systems. *The Journal of Chemical Physics* 115, 1716–1733.
- He, X., Lau, E.H., Wu, P., Deng, X., Wang, J., Hao, X., Lau, Y.C., Wong, J.Y., Guan, Y., Tan, X., et al., 2020. Temporal dynamics in viral shedding and transmissibility of COVID-19. *Nature Medicine* 26, 672–675.
- Jennness, S.M., Goodreau, S.M., Morris, M., 2018. EpiModel: an R package for mathematical modeling of infectious disease over networks. *Journal of statistical software* 84, 1–47.
- Keeling, M.J., Eames, K.T., 2005. Networks and epidemic models. *Journal of the Royal Society Interface* 2, 295–307.
- Keeling, M.J., Rohani, P., 2008. Modeling infectious diseases in humans and animals. Princeton university press.
- Kiss, I.Z., Miller, J.C., Simon, P.L., 2017. Mathematics of epidemics on networks: from exact to approximate models. Springer.

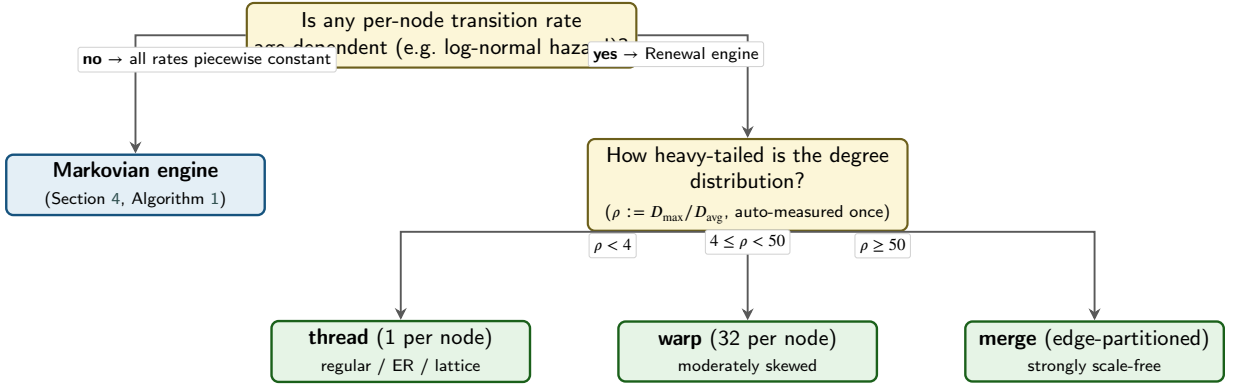
- Komarov, I., D'Souza, R.M., 2012. Accelerating the Gillespie exact stochastic simulation algorithm using hybrid parallel execution on graphics processing units. *PLoS One* 7, e46693.
- Lauer, S.A., Grantz, K.H., Bi, Q., Jones, F.K., Zheng, Q., Meredith, H.R., Azman, A.S., Reich, N.G., Lessler, J., 2020. The incubation period of coronavirus disease 2019 (COVID-19) from publicly reported confirmed cases: estimation and application. *Annals of Internal Medicine* 172, 577–582.
- Merrill, D., Garland, M., 2016. Merge-based parallel sparse matrix-vector multiplication, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, IEEE. pp. 1–12.
- Miller, J.C., Ting, T., 2020. Eon (epidemics on networks): a fast, flexible python package for simulation, analytic approximation, and analysis of epidemics on networks. *arXiv preprint arXiv:2001.02436*.
- Nobile, M.S., Cazzaniga, P., Besozzi, D., Mauri, G., 2014. cuTauLeaping: A GPU-powered tau-leaping stochastic simulator for massive parallel analyses of biological systems. *PLoS One* 9, e91963.
- Nowzari, C., Preciado, V.M., Pappas, G.J., 2016. Analysis and control of epidemics: A survey of spreading processes on complex networks. *IEEE Control Systems Magazine* 36, 26–46.
- NVIDIA Corporation, 2019. RAPIDS cuGraph: GPU-accelerated graph analytics. <https://github.com/rapidsai/cugraph>. Part of the RAPIDS open GPU data science suite.
- NVIDIA Corporation, 2024. CUDA C++ programming guide — section on CUDA graphs. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- Pastor-Satorras, R., Castellano, C., Van Mieghem, P., Vespignani, A., 2015. Epidemic processes in complex networks. *Reviews of Modern Physics* 87, 925–979.
- Perumalla, K.S., 2006. Discrete event execution alternatives on general purpose graphical processing units (GPGPUs), in: *Proceedings of the 20th Workshop on Principles of Advanced and Distributed Simulation*, IEEE. pp. 74–81.
- Preciado, V.M., Zargham, M., Enyioha, C., Jadbabaie, A., Pappas, G.J., 2014. Optimal resource allocation for network protection against spreading processes. *IEEE Transactions on Control of Network Systems* 1, 99–108.
- Rossetti, G., Milli, L., Rinzivillo, S., 2018. Ndlb: a python library to model and analyze diffusion processes over complex networks, in: *Companion Proceedings of the The Web Conference 2018*, pp. 183–186.
- Sahneh, F.D., Scoglio, C., Van Mieghem, P., 2013. Generalized epidemic mean-field model for spreading processes over multilayer complex networks. *IEEE/ACM Transactions on Networking* 21, 1609–1620.
- Sahneh, F.D., Vajdi, A., Shakeri, H., Fan, F., Scoglio, C., 2017. GEMFsim: A stochastic simulator for the generalized epidemic modeling framework. *Journal of Computational Science* 22, 36–44.
- Samaei, M.H., Sahneh, F.D., Scoglio, C., 2025. Fastgemf: Scalable high-speed simulation of stochastic spreading processes over complex multilayer networks. *IEEE Access*.
- Sanche, S., Lin, Y.T., Xu, C., Romero-Severson, E., Hengartner, N., Ke, R., 2020. High contagiousness and rapid spread of severe acute respiratory syndrome coronavirus 2. *Emerging infectious diseases* 26, 1470.
- Sherborne, N., Miller, J.C., Blyuss, K.B., Kiss, I.Z., 2018. Mean-field models for non-Markovian epidemics on networks. *Journal of Mathematical Biology* 76, 755–778.
- Starnini, M., Gleeson, J.P., Boguñá, M., 2017. Equivalence between non-Markovian and Markovian dynamics in epidemic spreading processes. *Physical Review Letters* 118, 128301.
- Vajdi, A., 2020. Stochastic spreading processes on networks. Kansas State University.
- Vajdi, A., Cohnstaedt, L.W., Noronha, L.E., Mitzel, D.N., Wilson, W.C., Scoglio, C.M., 2023. A non-markovian model to assess contact tracing for the containment of covid-19. *IEEE Transactions on Network Science and Engineering* 11, 197–211.
- Van Mieghem, P., van de Bovenkamp, R., 2013. Non-Markovian infection spread dramatically alters the susceptible-infected-susceptible epidemic threshold in networks. *Physical Review Letters* 110, 108701.
- Wang, Y., Davidson, A., Pan, Y., Wu, Y., Riffel, A., Owens, J.D., 2016. Gunrock: A high-performance graph processing library on the GPU, in: *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, ACM. pp. 1–12.
- Watkins, N.J., Nowzari, C., Preciado, V.M., Pappas, G.J., 2019. Robust economic model predictive control of continuous-time epidemic processes. *IEEE Transactions on Automatic Control* 65, 1116–1131.
- Wei, H., Yu, J.X., Lu, C., Lin, X., 2016. Speedup graph processing by graph ordering, in: *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*, ACM. pp. 1813–1828.
- Zou, P., Lv, Y.S., Deng, X.G., Ji, H., 2013. Epidemic simulation of large-scale social contact networks on GPU clusters, in: *Proceedings of the 2013 Winter Simulation Conference*, IEEE. pp. 1267–1278.



**Figure 1:** System overview. Every kernel reads from a shared GPU-resident state block (left). The Markovian engine (top) branches at runtime between a global rebuild (Control Mode, FlashNeighbor) and a sparse atomic update (Inertial Mode) depending on whether the number of per-step events is  $\ll N$ ; both modes converge on a common tau-leap Bernoulli sampler. The Renewal engine (bottom) is a single fused Triton kernel that streams CSR traversal, erfcx hazard evaluation, Bernoulli sampling + RNG, and the next-step infectivity write-back in one launch (Section 5.4). The dashed arc is the per-step buffer swap; next-step state becomes the next call's input.



**Figure 2:** SEIR model with mixed dynamics. The  $S \rightarrow E$  transition is edge-mediated and Markovian; the  $E \rightarrow I$  and  $I \rightarrow R$  transitions are nodal and non-Markovian, with age-dependent hazard functions  $h(\tau_i)$ .



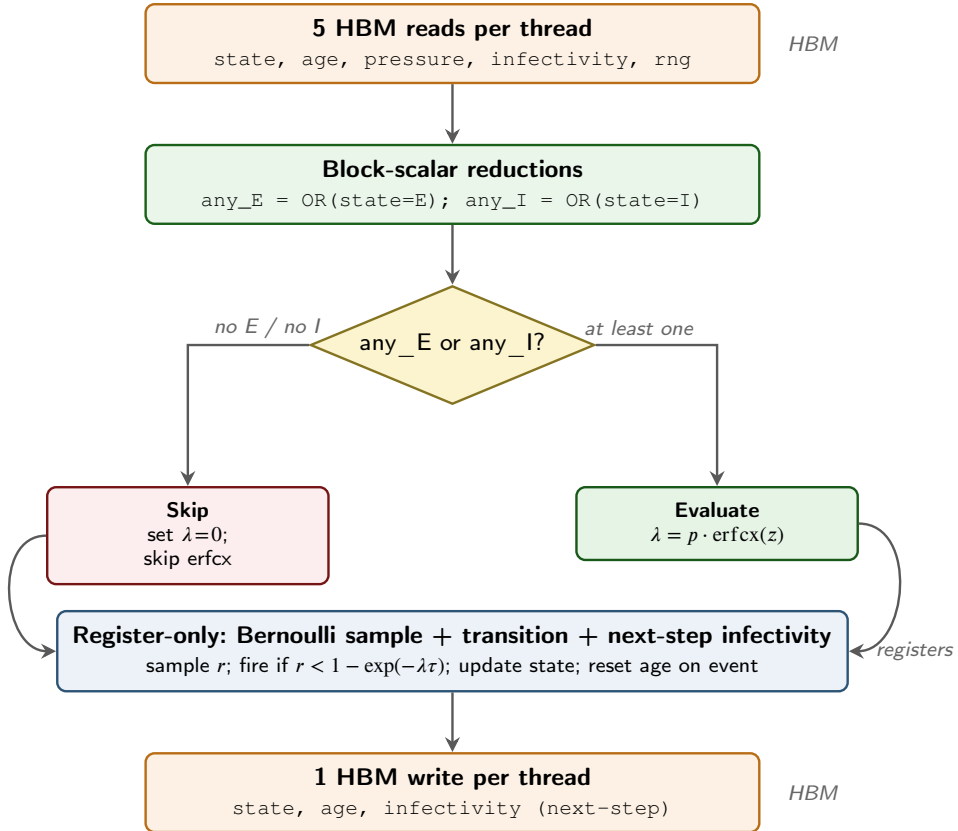
*Auto-dispatch (default): the engine reads `row_ptr` once at construction, computes  $\rho$ , and selects the strategy above. The user can override via `csr_strategy=...`*

**Figure 3:** Practitioner decision tree. A single top-level question (“is the hazard age-dependent?”) selects the engine; a single scalar ( $\rho = D_{\max}/D_{\text{avg}}$ , inspected once at construction) selects the CSR traversal strategy within the Renewal engine. Thresholds  $(\rho_w, \rho_m) = (4, 50)$  are calibrated in Appendix B.

```

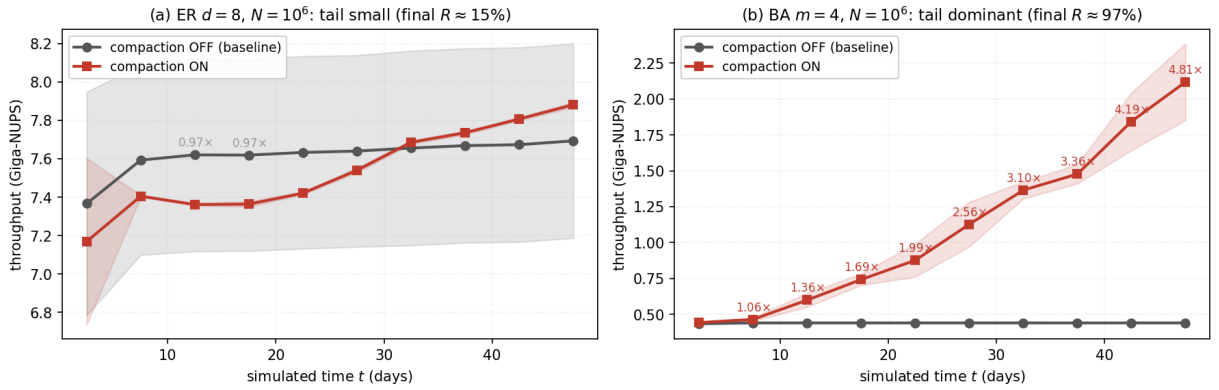
1 import torch
2 from flashspread import SEIRModel
3 from flashspread.core.network import FixedDegreeGraph
4 from flashspread.engines.renewal_fused import RenewalEngineFusedCUDAGraph
5
6 # 1. Graph and model are declarative:
7 graph = FixedDegreeGraph(num_nodes=1_000_000, degree=8, device="cuda")
8 model = SEIRModel(beta=0.25,
9                   mean_ei=5.0, median_ei=4.0,
10                  mean_ir=7.5, median_ir=5.0)
11 model.transmission_mode = "age_dependent" # alternative: "constant"
12
13 # 2. Engine picks the best CSR strategy from D_max / D_avg:
14 engine = RenewalEngineFusedCUDAGraph(
15     graph, model, device="cuda",
16     epsilon=0.03, tau_max=0.1, # tau-leaping knobs
17     csr_strategy="auto", # thread / warp / merge / auto
18     steps_per_launch=50, # CUDA Graph batch size b
19     seed=12345,
20 )
21 engine.seed_infection(num_infected=100, state=model.exposed)
22
23 # 3. Run:
24 while engine.current_time < 50.0:
25     _, state = engine.step()
26
27 print(engine.count_by_state()) # [S, E, I, R] on device
28 # Expected ensemble-mean populations at t=50 on this benchmark:
29 # roughly [~0.04, ~0.02, ~0.05, ~0.89] * num_nodes
30 # (useful as a one-line post-install sanity check; variance across
31 # seeds is a few percent, dominated by the synchronous-update floor
32 # discussed in Appendix C.)
  
```

**Listing 1:** Declarative simulation setup with FLASHSPREAD. The engine is constructed once with a graph, a compartment model, and a tolerance; subsequent `step()` calls advance one batched CUDA Graph replay. Both the CSR strategy (“auto” / “thread” / “warp” / “merge”) and the model’s `transmission_mode` (“constant” Markovian edges vs. “age\_dependent” source-node shedding) are runtime choices, not recompiles.

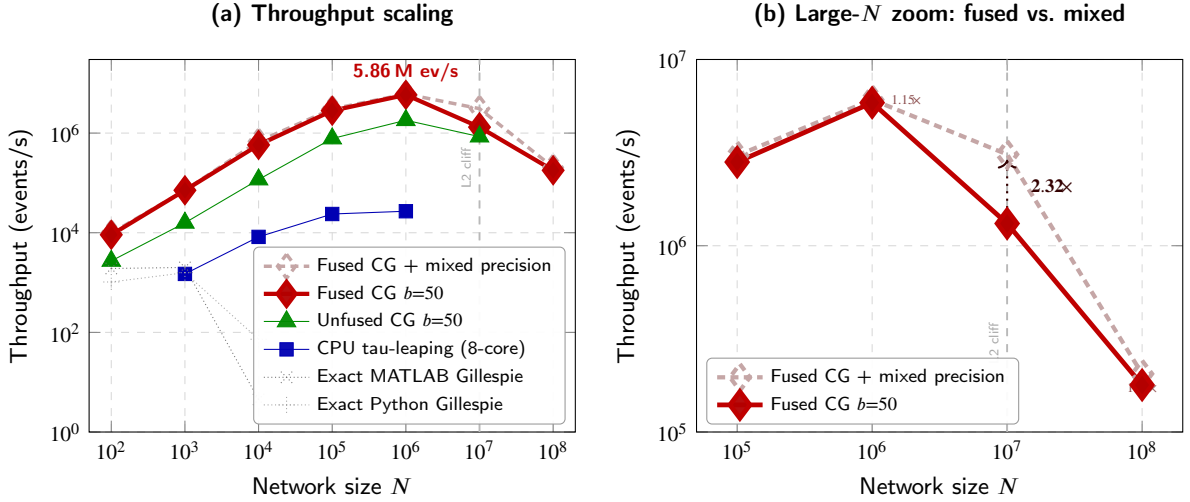


**Figure 4:** Per-thread control flow inside Algorithm 3: five coalesced HBM reads on entry, a two-way branch selected by block-scalar reductions over the current compartments in the thread block, register-only work (Bernoulli sample, transition, age reset, next-step infectivity), and one coalesced HBM write on exit. The branch is *block-level* (a scalar decision for the whole warp/block), not per-lane: blocks with no E or I node skip the  $\sim 55$ -FLOP  $\text{erfcx}$  call entirely without breaking CUDA Graph capture, while blocks with at least one E or I node evaluate the hazard for all lanes and mask-select the result. Companion to the data-layout diagram of Figure 14 and the memory-hierarchy diagram of Figure 16.

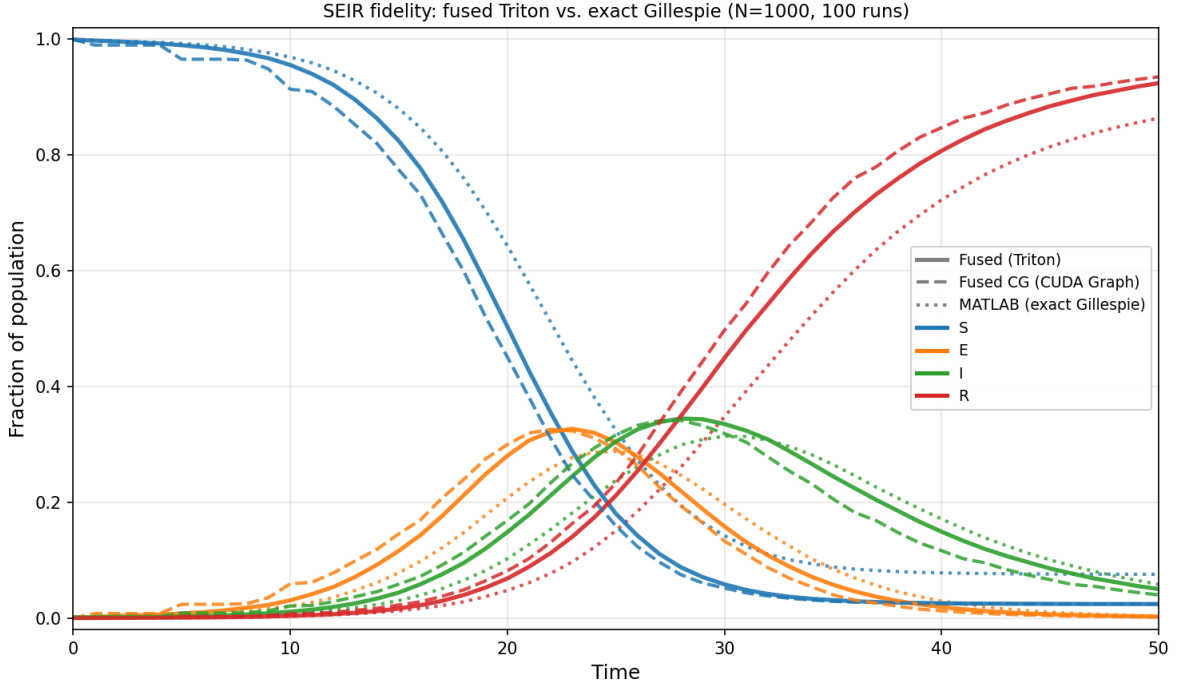


Active-node compaction: NUPS( $t$ ) vs epidemic phase

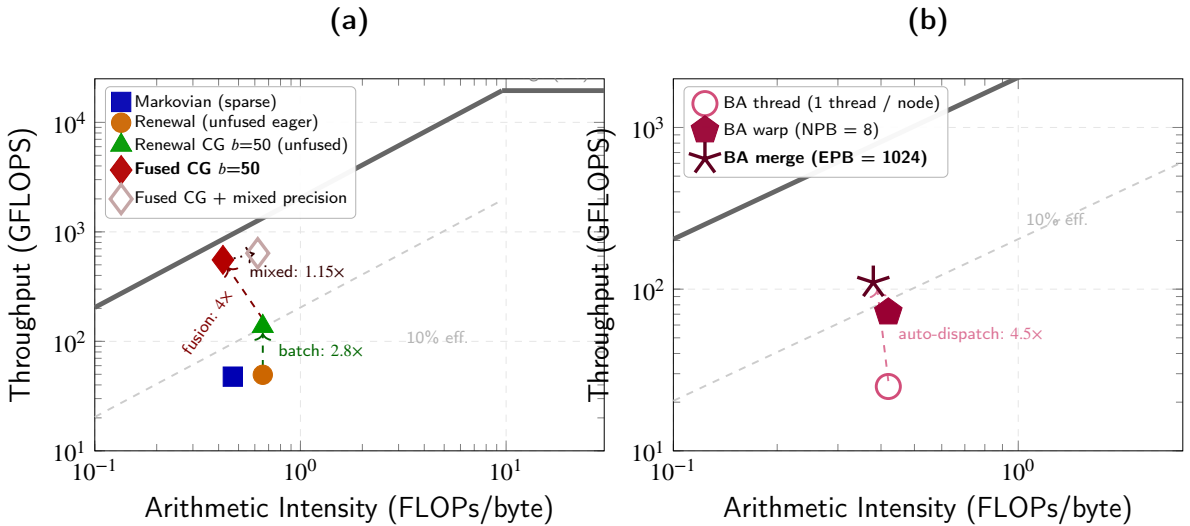
**Figure 5:** Active-node compaction: NUPS( $t$ ) stratified into ten temporal windows of the TF = 50 renewal benchmark, 5 replicate trials (mean  $\pm$  1 std shaded). Left: ER  $d=8$ ,  $N=10^6$ , where the epidemic only reaches  $\approx 15\%$  attack rate within TF=50; compaction is  $\sim 3\%$  slower than baseline during the takeoff and peak windows (refresh overhead exceeds marginal skip gain) and crosses over to a small lift only in the last temporal bucket. Right: BA  $m=4$ ,  $N=10^6$ , where explosive hub-driven takeoff reaches  $\approx 97\%$  attack rate; the shrinking active set drives a progressive speedup from 1.0x (takeoff) to 4.8x (last bucket, num\_active  $\approx 3\%N$ ), giving 1.53x on the whole-run mean (Table 3). The topology-dependent pattern makes compaction a *high-saturation / long-horizon* optimisation: it is most valuable on scale-free networks, policy evaluations that simulate past the peak, and reinforcement-learning rollouts that run to extinction.



**Figure 6:** Renewal engine throughput (SEIR, log-normal,  $d=8$ ) on Erdős-Rényi networks. Panel (a) shows the full scaling range  $N \in [10^2, 10^8]$  with six curves: Fused CG  $b=50$  (red, headline), Fused CG  $b=50$  with mixed-precision storage (pink dashed; Section 5.7), unfused CUDA-Graph  $b=50$ , an 8-core CPU tau-leaping baseline (derived from NUPS via the shared  $7.24 \times 10^{-4}$  events-per-NUPS ratio calibrated at  $N = 10^6$ ; a property of the algorithm, not the hardware), and two exact event-driven references from the Scoglio et al. suite (MATLAB and Python). Panel (b) zooms into  $N \in [10^5, 10^8]$  and keeps only the two headline curves so the L2-cliff region is legible. The fused CUDA-Graph engine reaches 5.86 M events/s at  $N = 10^6$  (equivalently 8.09 Giga-NUPS;  $217\times$  over the CPU tau-leaping baseline at the same  $N$ ). The vertical gap between the red (Fused CG) and green (unfused CG) curves at  $N = 10^6$  is the  $3.24\times$  fusion gain in events/s (equivalently  $4.3\times$  in NUPS; see Table 6), isolating the benefit of collapsing the CSR + hazard + Bernoulli + infectivity write-back pipeline into a single Triton launch from the separate batching benefit of CUDA Graph capture. At  $N = 10^7$ , where the fp32 CSR + state working set exceeds the A100's 40 MB L2 cache, the baseline collapses to 1.33 M events/s while mixed precision holds at 3.10 M events/s, a  $2.32\times$  lift; the narrower per-node dtypes reduce the per-step working-set footprint by  $\approx 3\times$  and effectively push the L2 cliff out by the same factor. At  $N = 10^8$  both configurations are deep in HBM-bandwidth-bound execution (working sets  $\sim 10$  GB and  $\sim 6$  GB respectively, both  $\gg 40$  MB L2) and the gain collapses back toward the bandwidth-ratio asymptote at  $1.15\times$ . The mechanism of the cliff is analysed in Appendix D.3 and Figure 16. The earlier GPU-unfused-eager curve is omitted from this figure because it is a pedagogical pre-CUDA-Graph baseline whose contribution is already captured by the CPU-to-CG gap and discussed in prose (Section 5.4). The repository README lists the measurement scripts and CSV artefacts for each curve.

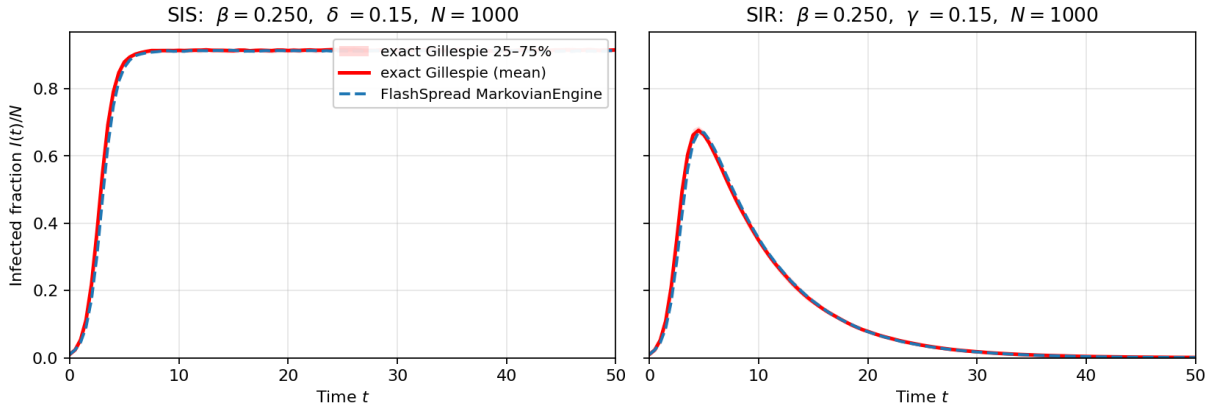


**Figure 7:** Fidelity validation. Fused GPU kernel (solid), fused CUDA Graph (dashed), and exact non-Markovian Gillespie (dotted) on the validation benchmark: Erdős-Rényi graph with  $N = 10^3$  nodes and average degree  $d = 8$ ;  $\beta = 0.25$ ; log-normal  $E \rightarrow I$  with mean 5.0 d and median 4.0 d; log-normal  $I \rightarrow R$  with mean 7.5 d and median 5.0 d; 100 independent Monte Carlo runs. The residual gap visible against the exact reference is the structural-bias floor of synchronous Bernoulli updates on a network, quantified in detail in Appendix C, Figure 11.



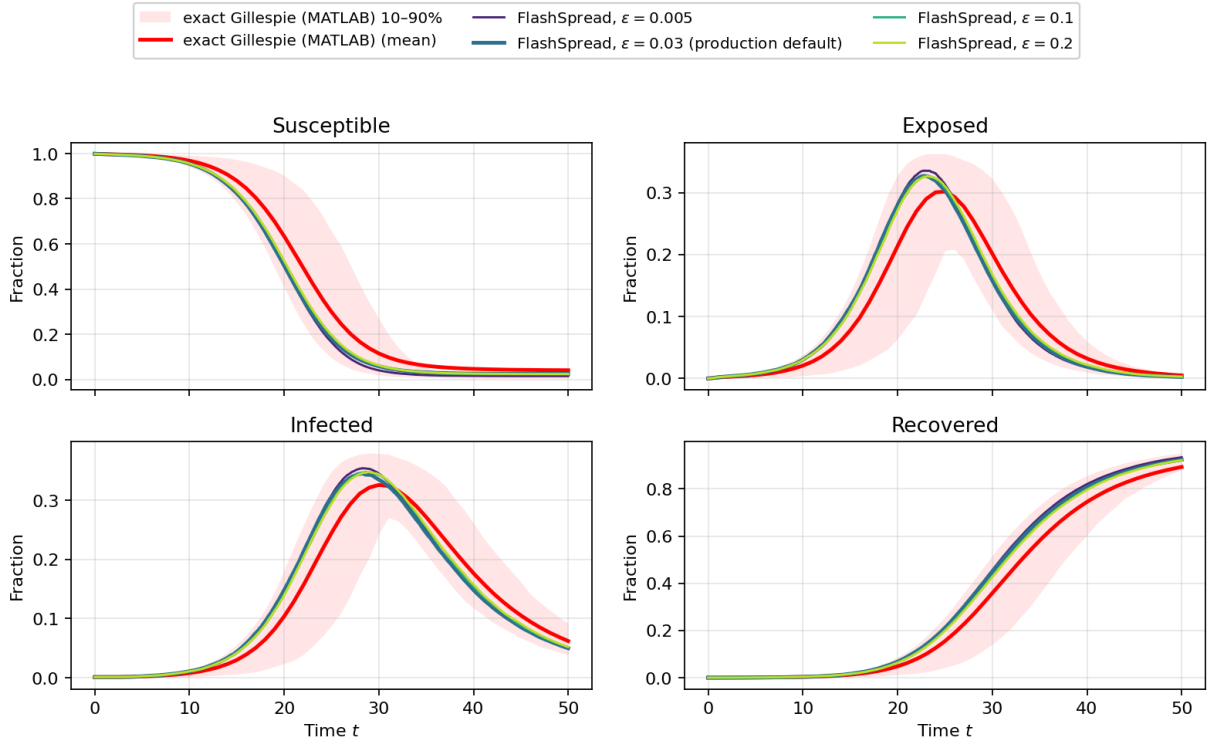
**Figure 8:** Roofline analysis on NVIDIA A100 (peak 19.5 TFLOPS FP32 / 2039 GB/s HBM, ridge =  $19500/2039 \approx 9.56$  FLOPs/byte). In panel (a), kernel fusion shifts the operating point upward by 4x; CUDA Graph batching adds 2.8x. Enabling mixed-precision storage on top of Fused CG (Section 5.7) shifts the point up-and-right by the  $\sim 32\%$  per-step byte reduction: the AI grows from 0.42 to  $\approx 0.62$  (same FLOPs, fewer bytes), and the achieved throughput grows by the measured 1.15x (from 555 to 638 GFLOPS). All configurations lie firmly in the memory-bound regime (left of the ridge), but mixed precision moves the kernel closer to compute-bound territory, which is why the incremental throughput win is less than the 1.32x memory-bytes ratio would predict. Panel (b) shows the CSR-strategy dispatch on BA  $m=4$  at the same scale.

FlashSpread Markovian engine vs. exact Doob-Gillespie (100 Monte Carlo runs per curve)

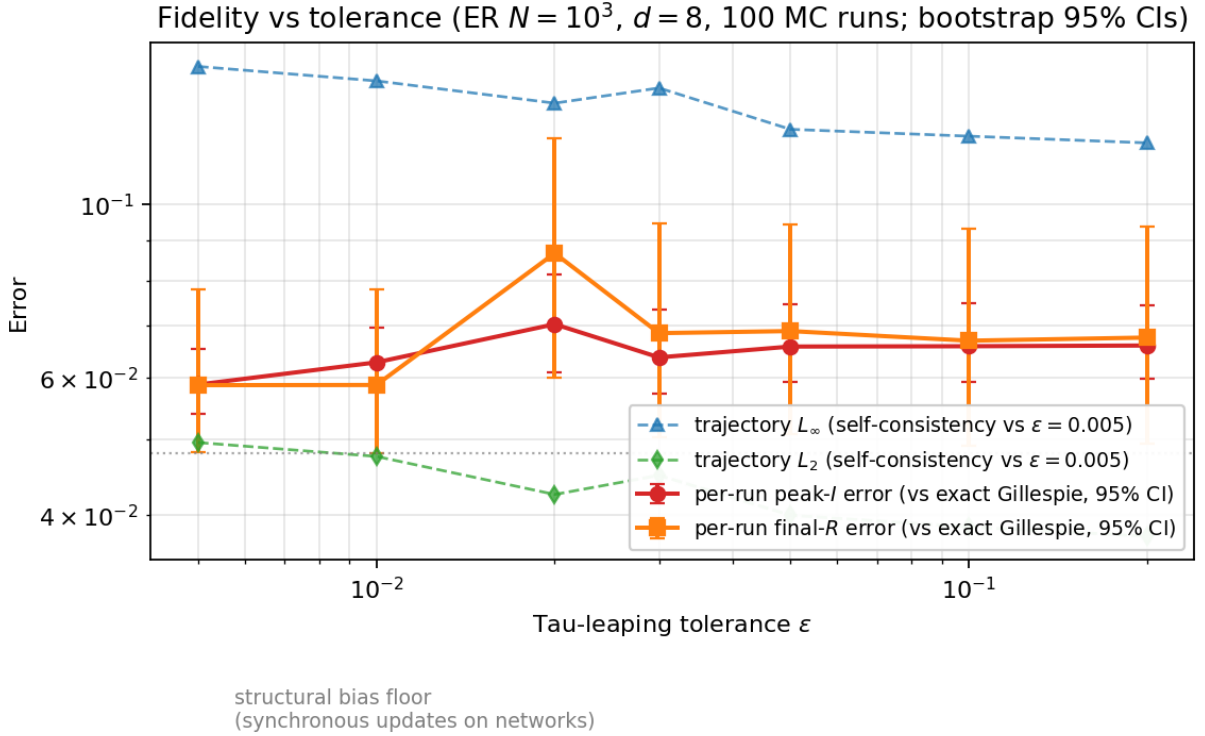


**Figure 9:** FLASHSPREAD Markovian engine (blue dashed) versus exact Doob-Gillespie (red solid, with 25–75% quantile band) on ER  $d = 8$ ,  $N = 10^3$ ,  $\beta = 0.25$ , recovery rate 0.15. Left: SIS (endemic steady state). Right: SIR (single epidemic wave). 100 independent Monte Carlo runs per curve. The tau-leaping curve tracks the exact reference across the full trajectory on both models, confirming that the Markovian engine's  $\varepsilon$ -selected step sampler operates correctly in both the sparse-event regime (early-time SIS and the tail of SIR) and the dense-event regime (endemic SIS and the SIR peak).

Ensemble-mean SEIR compartments: FlashSpread (tau-leaping) vs. exact non-Markovian Gillespie

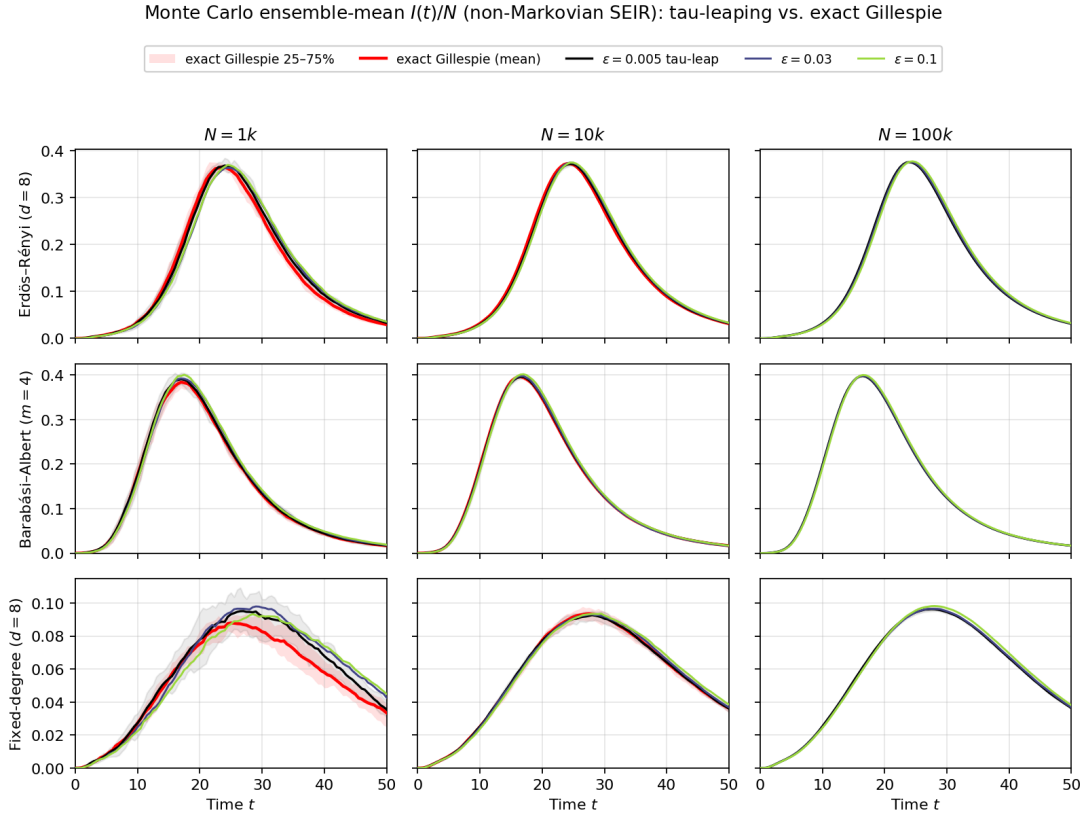


**Figure 10:** Ensemble-mean SEIR trajectories overlaid on the exact non-Markovian Gillespie reference (red solid, with 10–90% Monte Carlo quantile band), plus four tau-leaping tolerances, on the validation benchmark (Erdős–Rényi graph with  $N = 10^3$  nodes and average degree  $d = 8$ ;  $\beta = 2/d = 0.25$ ; log-normal  $E \rightarrow I$  and  $I \rightarrow R$  with the parameters of Section 6; 100 independent Monte Carlo trajectories per curve). The exact reference is the companion MATLAB implementation of the non-Markovian Gillespie algorithm (Boguñá et al., 2014). The  $\epsilon = 0.005$  tau-leaping curve (black) is the practical discretization limit; coarser  $\epsilon$  deviates smoothly but tracks the exact curve tightly. The residual offset between every tau-leaping curve and the exact reference is the structural bias of synchronous Bernoulli updates on a network, independent of  $\epsilon$ .



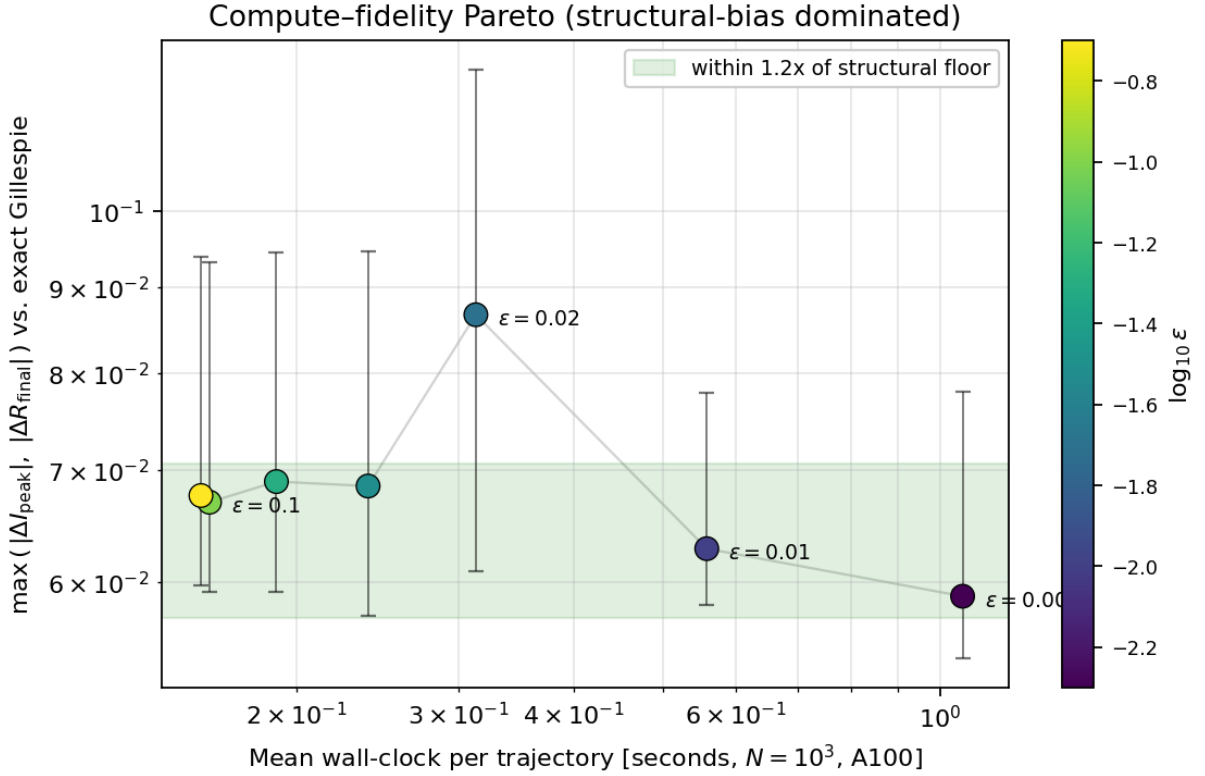
**Figure 11:** Error decomposition for Bernoulli tau-leaping, with bootstrap 95% confidence intervals over the 100-run ensemble. Solid markers (with error bars) are per-run absolute errors against the exact non-Markovian Gillespie reference produced by the companion MATLAB simulator. Dashed markers are self-consistency trajectory-level errors against the finest  $\varepsilon = 0.005$  tau-leaping ensemble. The solid curves' CIs overlap and sit at the structural bias floor (dotted grey), showing that within ensemble noise the peak- $I$  and final- $R$  errors do not detectably decrease below  $\sim 6\%$  as  $\varepsilon \rightarrow 0$  across the sweep; the dashed curves continue decreasing as pure discretization error vanishes. Below  $\varepsilon \approx 0.1$  the structural term dominates and further reductions in  $\varepsilon$  buy no detectable fidelity improvement on this benchmark.



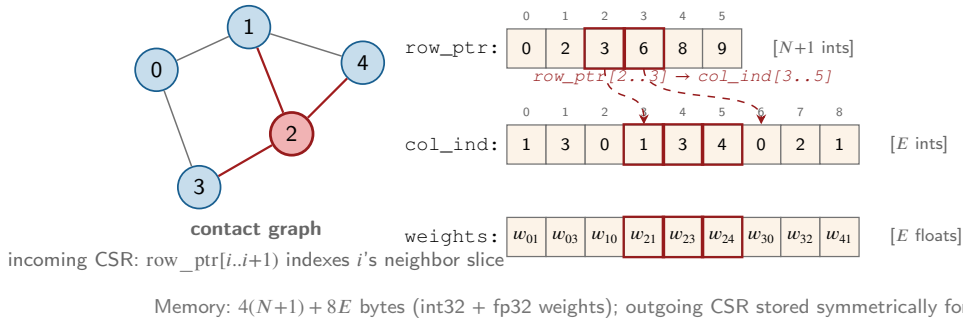


(20 trajectories per  $(G, N, \varepsilon)$  for tau-leaping and per  $(G, N)$  for exact; initial seed =  $\max(10, 0.01N)$  Exposed nodes;  $\beta = 2/\bar{d} = 0.25$ ; log-normal  $E \rightarrow I$ ,  $I \rightarrow R$  with the main-text parameters.)

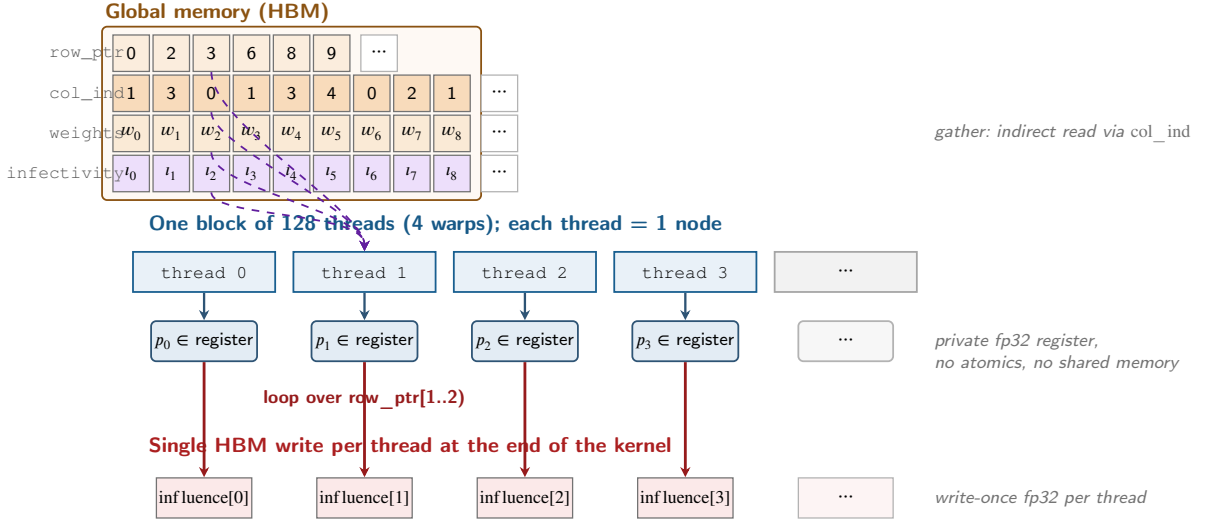
**Figure 12:** Monte Carlo ensemble-mean  $I(t)/N$  for the non-Markovian SEIR model (log-normal  $E \rightarrow I$  and  $I \rightarrow R$ ) across three graph topologies (rows) and three sizes (columns). Each panel overlays an exact non-Markovian Gillespie reference (red, with 25–75% inter-quartile band shaded; implemented via our companion CPU simulator, cited in the repository README) and tau-leaping at  $\varepsilon \in \{0.005, 0.03, 0.1\}$  (the  $\varepsilon = 0.005$  curve is the discretization-limit reference in black). Exact reference curves are provided for  $N \in \{10^3, 10^4\}$ ;  $N = 10^5$  is beyond the CPU-Gillespie budget for 20 trials but is covered by tau-leaping alone. Every epidemic takes off (initial seed of  $\max(10, 0.01N)$  Exposed nodes) and the tau-leaping curves are indistinguishable from the exact Gillespie curve at plot resolution across all covered cells, demonstrating that the fidelity story established at  $N = 10^3$  on ER transfers directly to scale-free and homogeneous topologies and to sizes up to  $10^5$ .



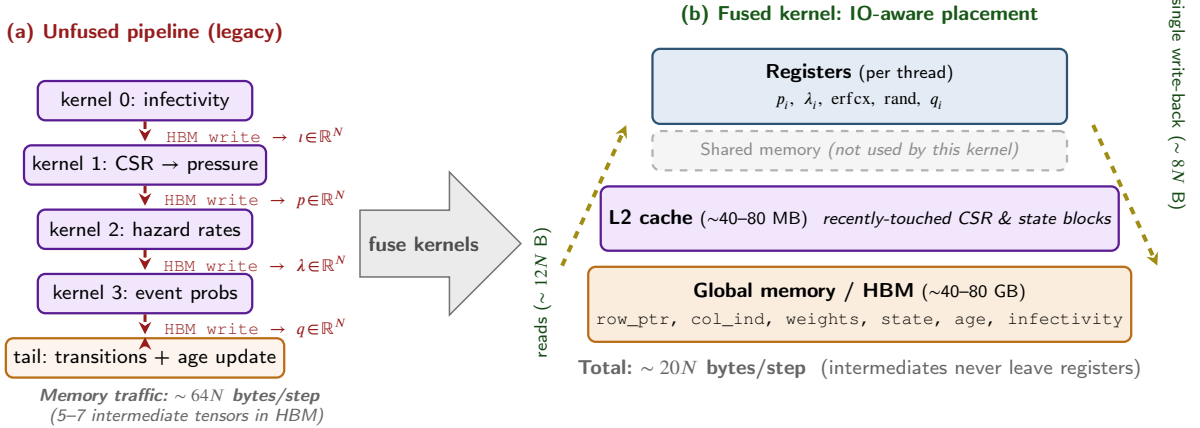
**Figure 13:** Wall-clock–fidelity Pareto. Each point is one  $\epsilon$  value (color:  $\log_{10} \epsilon$ , labelled). The vertical axis is  $\max(|\Delta I_{\text{peak}}|, |\Delta R_{\text{final}}|)$  — the larger of the two per-run absolute errors against the exact Gillespie reference — chosen so that no metric can hide a worse metric. Error bars on both axes are 95% bootstrap CIs (1,000 resamples over the 100-run ensemble): horizontal bars for wall-clock, vertical bars for the fidelity metric. The vertical CIs overlap across two decades of  $\epsilon$  while wall-clock spans a factor of seven: above the fast-end cluster, additional compute does not translate into statistically detectable fidelity gains.



**Figure 14:** CSR layout used throughout FLASHSPREAD. Left: a small undirected contact graph; node 2 (red) is the focus. Right: the three CSR arrays. `row_ptr[i..i+1]` delimits node  $i$ 's neighbor slice in `col_ind` and `weights`. Reading node 2's neighbors therefore costs two coalesced scalar loads (pointer endpoints) plus a contiguous burst of  $D_2$  int/float pairs. Incoming and outgoing CSRs are stored separately: the incoming copy drives both engines' FlashNeighbor reads; the outgoing copy drives Inertial-Mode sparse atomic propagation when a single node transitions.



**Figure 15:** FlashNeighbor kernel, per-thread view. Each thread owns a unique target node  $i$  and accumulates its influence  $p_i$  in a private register. The loop body reads  $\text{row\_ptr}[i..i+1)$  once (coalesced), then iterates the neighbor slice: each iteration issues one indirect gather into  $\text{col\_ind}$ , one coalesced load of weights, and one indirect gather into the infectivity buffer (the L2 path is the only place these dashed loads land because they are node-indirect). The accumulation  $p_i \leftarrow p_i + w_{ij} t_{cij}$  lives entirely in the thread's register file; the kernel never touches shared memory. At the end of the loop the thread performs exactly one fp32 store back to HBM for its node. No atomic operations are required because each target  $i$  is uniquely owned.



**Figure 16:** GPU memory-hierarchy flow, before and after fusion. (a) The legacy PyTorch/Triton pipeline launches five kernels per step; each kernel reads from HBM and writes its  $\mathcal{O}(N)$  intermediate (infectivity, pressure, rates, event probabilities, event mask) back to HBM before the next kernel starts. Total traffic is  $\sim 64N$  bytes per step. (b) The fused kernel keeps all of these intermediates in per-thread registers; only the inputs (state, age, infectivity of the current step) travel HBM  $\rightarrow$  L2  $\rightarrow$  register, and only the final per-node outputs travel back to HBM. Shared memory is deliberately unused on this kernel: the per-node computation has no cross-thread data dependency inside a block (each thread owns one target node), so staging through shared memory would add latency without reducing traffic. Net traffic drops to  $\sim 20N$  bytes per step, matching the memory-bandwidth-limited throughput reported in Figure 8.

